

# **END-TO-END 2D BUILDING POLYGON DELINEATION FROM NORMALIZED AIRBORNE LIDAR POINT CLOUDS: A DEEP LEARNING APPROACH**

MUHAMMAD HASHIR BAIG

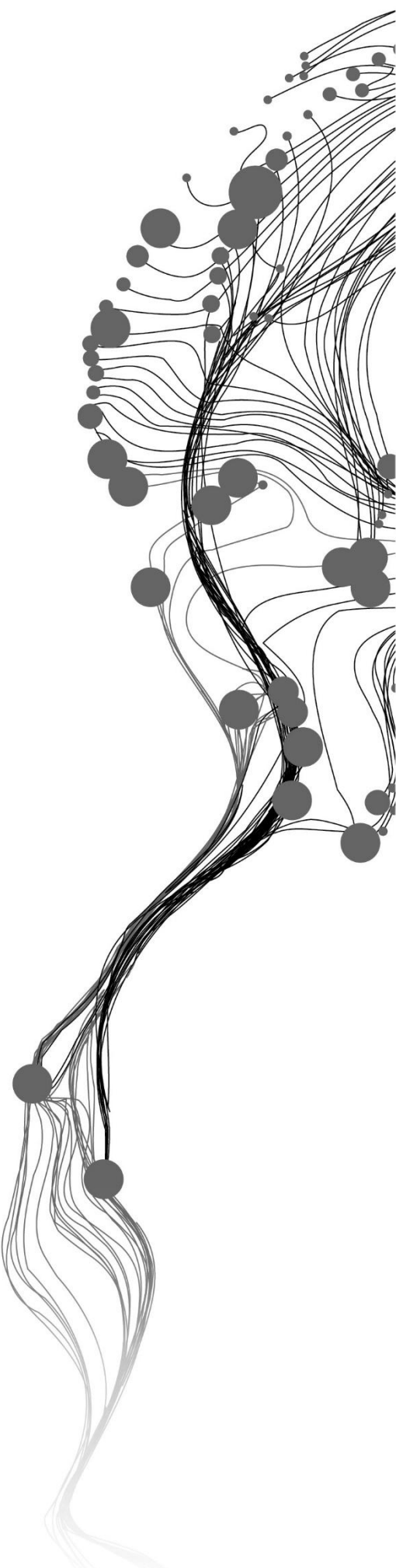
June, 2026

SUPERVISORS:

dr.ir. S.J. Oude Elberink

prof.dr.ing. C. Persello





# **END-TO-END 2D BUILDING POLYGON DELINEATION FROM NORMALIZED AIRBORNE LIDAR POINT CLOUDS: A DEEP LEARNING APPROACH**

**MUHAMMAD HASHIR BAIG**

Enschede, The Netherlands, June, 2026

Thesis submitted to the Faculty of Geo-Information Science and Earth  
Observation of the University of Twente in partial fulfilment of the  
requirements for the degree of Master of Science in Geo-information Science  
and Earth Observation.

Specialization: Geoinformatics

## **SUPERVISORS:**

dr.ir. S.J. Oude Elberink

prof.dr.ing. C. Persello

## **THESIS ASSESSMENT BOARD:**

prof.dr.ir. M.G. Vosselman (Chair)

dr. R.C. Lindenbergh (External Examiner, TU Delft)

#### DISCLAIMER

This document describes work undertaken as part of a programme of study at the Faculty of Geo-Information Science and Earth Observation of the University of Twente. All views and opinions expressed therein remain the sole responsibility of the author, and do not necessarily represent those of the Faculty.

# ABSTRACT

Accurate 2D building outlines are essential for cadastral mapping and urban applications, yet most deep learning methods for airborne LiDAR target 3D roof or wireframe reconstruction, while raster-based pipelines require post-hoc vectorization and regularization, and traditional  $\alpha$ -shape methods depend on hand-tuned, density-sensitive parameters. This thesis investigates whether a graph-based learning framework can recover topologically consistent, closed 2D building polygons directly from normalized airborne LiDAR point clouds. The Point2Roof framework, originally developed for 3D roof-graph reconstruction, is adapted for planimetric outline delineation. Its central modification replaces the original pairwise edge classifier with a phase prediction head that assigns each candidate vertex a position along the canonical boundary, so that the outline is recovered by sorting per-vertex angular positions rather than by classifying vertex pairs. The model is trained and evaluated on pre-segmented building components from the Dutch AHN dataset, supervised by BAG cadastral footprints over Enschede.

On a held-out test set, the predicted outlines match the BAG references closely on average (IoU 0.862, F1 0.888, Boundary IoU 0.783), and every produced polygon is consistent by construction, with a prediction-failure rate of about 2%. The principal limitation is vertex completeness: the polygons recover roughly 84% of reference corners on average, because weakly supported corners are missed during classification or absorbed during clustering. A controlled synthetic-occlusion experiment shows that reduced boundary coverage sharply lowers recall and corner recovery while leaving topological validity intact. The work contributes not a new network but the adaptation, supervision design and critical failure-mode analysis of a graph-based LiDAR framework, yielding outlines suitable for human-supervised cadastral proposal rather than fully automated entry.

**Keywords:** 2D building outline delineation, Airborne LiDAR, Point cloud deep learning, Point2Roof, Phase prediction head, Topological validity, Cadastral mapping, Basisregistratie Adressen en Gebouwen (BAG)

# ACKNOWLEDGEMENTS

First and foremost, I am deeply thankful to Almighty Allah, who granted me the ability, strength, and opportunity to pursue and complete this work. Without His countless blessings and guidance, none of this would have been possible.

This thesis would not have been possible without the guidance, support, and encouragement of many people, to whom I am deeply grateful.

I would like to express my sincere gratitude to my first supervisor, Sander Oude Elberink. His guidance shaped this research from beginning to end. At every stage of the work he generously shared his deep knowledge and experience and his thoughtful, patient feedback consistently steered me in the right direction. Whenever I lost sight of the larger picture or drifted from the central question, he brought me back onto the right path. I am especially thankful for the time he invested in me and for the care with which he challenged and sharpened my thinking; this thesis owes a great deal to his mentorship. I would also like to thank my second supervisor, Claudio Persello, for his valuable insights and constructive feedback. His perspective and expertise helped refine the direction of this research and strengthened the quality of the final work.

My heartfelt thanks go to my family and friends, whose unconditional support has been a constant source of strength. My friends; Uzziel, Amin, Mujtaba, Taimoor, Ammar, Ahmad, Hamza, Tishya, Yumnah and Liangwe Chen. Their encouragement carried me through the difficult moments and made this entire journey worthwhile. I am profoundly grateful for their patience, their belief in me, and their unwavering presence throughout this period.

Finally, I would like to thank my friends at ITC, and in particular my fellow batchmates of the Geoinformatics specialization. The shared experiences, late discussions and companionship made this journey not only intellectually rewarding but also genuinely enjoyable and the friendships formed here are ones I will carry with me for years to come.

# TABLE OF CONTENTS

---

|                                                               |    |
|---------------------------------------------------------------|----|
| List of figures .....                                         | v  |
| List of tables .....                                          | vi |
| 1. Introduction. ....                                         | 7  |
| 1.1. Background. ....                                         | 7  |
| 1.2. Problem Statement . ....                                 | 11 |
| 1.3. Research Gap. ....                                       | 11 |
| 2. Objectives and Research Questions. ....                    | 12 |
| 2.1. Main Objective .....                                     | 12 |
| 2.2. Subobjectives and Research Questions .....               | 12 |
| 3. Materials and Methods. ....                                | 13 |
| 3.1. Datasets and Technologies .....                          | 13 |
| 3.2. Methodology . ....                                       | 14 |
| 3.2.1. Data Preprocessing .....                               | 14 |
| 3.2.2. Training Sample Label Generation .....                 | 15 |
| 3.2.3. Model Architecture .....                               | 19 |
| 3.2.4. Detailed Working of Adapted Point2Roof Framework. .... | 23 |
| 3.2.5. Configuration and Hyperparameters .....                | 30 |
| 3.2.6. Model Training and Data Split Strategy .....           | 31 |
| 3.2.7. Model Inference and Polygon Generation .....           | 32 |
| 3.2.8. Evaluation Metrics . ....                              | 32 |
| 3.2.9. Comparative Evaluation with Poly R-CNN. ....           | 33 |
| 4. Results and Discussion. ....                               | 33 |
| 4.1. Training Details .....                                   | 33 |
| 4.2. Training Convergence . ....                              | 34 |
| 4.3. Quantitative Analysis. ....                              | 35 |
| 4.3.1. Evaluation Setup and Metric Summary .....              | 35 |
| 4.3.2. Geometric Accuracy (RQ 2.1) .....                      | 36 |
| 4.3.3. Cadaster Quality Requirements (RQ 2.2) .....           | 37 |
| 4.3.4. Summary. ....                                          | 37 |
| 4.4. Success Cases. ....                                      | 38 |
| 4.4.1. Standalone Building Components. ....                   | 38 |
| 4.4.2. Large Building Components .....                        | 41 |
| 4.5. Failure Cases. ....                                      | 44 |
| 4.5.1. Cluster Absorption in Adjacent Corners .....           | 44 |
| 4.5.2. Spurious Vertices .....                                | 46 |
| 4.5.3. Ordering Failure (phase head) .....                    | 46 |
| 4.5.4. Compound Failure. ....                                 | 48 |
| 4.6. Robustness Evaluation. ....                              | 49 |
| 5. Research Challenges. ....                                  | 54 |
| 6. Limitations. ....                                          | 54 |
| 7. Future Work. ....                                          | 55 |
| 8. Conclusion. ....                                           | 56 |
| 9. Annexes. ....                                              | 60 |

List of References.....61



## LIST OF FIGURES

---

|                                                                                                                                                                                                                                                                                                                                                                                                                                                        |    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 1 Point-based normalization of Point Cloud for a portion of Enshede City, Netherlands .....                                                                                                                                                                                                                                                                                                                                                     | 7  |
| Figure 2 Building outline determination initiated by mean shift segmentation (a) and (c) and planar face extraction (b). The points assigned to one building are shown as red circles in (c) and as magenta crosses in (d) and (f). The generalization of the 2D-shape (cyan polygon in (d)) is applied using an angular criterion (e). The generalized (green) and the regularized outline (black) are shown in (f). (Dorninger & Pfeifer, 2008)..... | 9  |
| Figure 3 Illustration of an example of BAG polygon in an urban setting of Enshede City, Netherlands...                                                                                                                                                                                                                                                                                                                                                 | 10 |
| Figure 4 Illustration of an example building instance component point cloud.....                                                                                                                                                                                                                                                                                                                                                                       | 10 |
| Figure 5 Illustration of BAG polygons (converted to object wavefront format) spatially mapped to corresponding components (standalone case) .....                                                                                                                                                                                                                                                                                                      | 15 |
| Figure 6 Building instance components extraction from point clouds via connected components .....                                                                                                                                                                                                                                                                                                                                                      | 16 |
| Figure 7 Annotation file (Object wavefront file).....                                                                                                                                                                                                                                                                                                                                                                                                  | 16 |
| Figure 8 Training Sample Pair Visual.....                                                                                                                                                                                                                                                                                                                                                                                                              | 16 |
| Figure 9 Sequential structuring of each Training Sample Pair Visual .....                                                                                                                                                                                                                                                                                                                                                                              | 16 |
| Figure 10 Accepted sample for training (Green). Excluded samples; (Pink) Imperfect overlap due to possible occlusion; Point cloud overshooting, imperfect overlap (Red) .....                                                                                                                                                                                                                                                                          | 17 |
| Figure 11 Illustration of training sample augmentation .....                                                                                                                                                                                                                                                                                                                                                                                           | 17 |
| Figure 12 Phase head module architecture.....                                                                                                                                                                                                                                                                                                                                                                                                          | 21 |
| Figure 13 Point2Roof 2D outline delineation adapted workflow illustration .....                                                                                                                                                                                                                                                                                                                                                                        | 23 |
| Figure 14 Candidate corner points binary classification representation .....                                                                                                                                                                                                                                                                                                                                                                           | 24 |
| Figure 15 Candidate corner points offset regression representation.....                                                                                                                                                                                                                                                                                                                                                                                | 25 |
| Figure 16 Centroid Generation via DBSCAN representation .....                                                                                                                                                                                                                                                                                                                                                                                          | 26 |
| Figure 17 Phase head's output representation .....                                                                                                                                                                                                                                                                                                                                                                                                     | 28 |
| Figure 18 Graph flattening representation .....                                                                                                                                                                                                                                                                                                                                                                                                        | 29 |
| Figure 19 Polygon creation representation .....                                                                                                                                                                                                                                                                                                                                                                                                        | 29 |
| Figure 20 Training and Testing Data Split for Enschede Municipality.....                                                                                                                                                                                                                                                                                                                                                                               | 31 |
| Figure 21 AHN 6 Tile (connected components) Spatially clipped to MapLmLas Tile extent .....                                                                                                                                                                                                                                                                                                                                                            | 32 |
| Figure 22 BAG polygons overlaid over MapLmLas training & testing tiles (Blue: training & Red: testing)                                                                                                                                                                                                                                                                                                                                                 | 32 |
| Figure 23 Training losses curves for binary classification, offset regression, refinement and phase prediction heads .....                                                                                                                                                                                                                                                                                                                             | 34 |
| Figure 24 Successful outline delineation of building components .....                                                                                                                                                                                                                                                                                                                                                                                  | 41 |
| Figure 25 Successful outline delineation large standalone building component.....                                                                                                                                                                                                                                                                                                                                                                      | 42 |
| Figure 26 Another successful case of larger component instance.....                                                                                                                                                                                                                                                                                                                                                                                    | 43 |
| Figure 27 Failure case (vertex positioning).....                                                                                                                                                                                                                                                                                                                                                                                                       | 44 |
| Figure 28 Other Cluster absorption cases .....                                                                                                                                                                                                                                                                                                                                                                                                         | 45 |
| Figure 29 Spurious vertices case.....                                                                                                                                                                                                                                                                                                                                                                                                                  | 46 |
| Figure 30 Failure case (wrong vertex ordering and cluster absorption) .....                                                                                                                                                                                                                                                                                                                                                                            | 47 |
| Figure 31 Compound Failure case (wrong vertex ordering and cluster absorption).....                                                                                                                                                                                                                                                                                                                                                                    | 48 |
| Figure 32 Illustration of disc-based occlusion model (created with assistance of AI) .....                                                                                                                                                                                                                                                                                                                                                             | 50 |
| Figure 33 Illustrations of synthetically added occlusion, including inference.....                                                                                                                                                                                                                                                                                                                                                                     | 51 |

# LIST OF TABLES

---

Table 1 Comparison between Paired Point Attention Module and Phase Head.....20

Table 2 Model configurations and parameters .....30

Table 3 Quantitative evaluation of inferred samples.....35

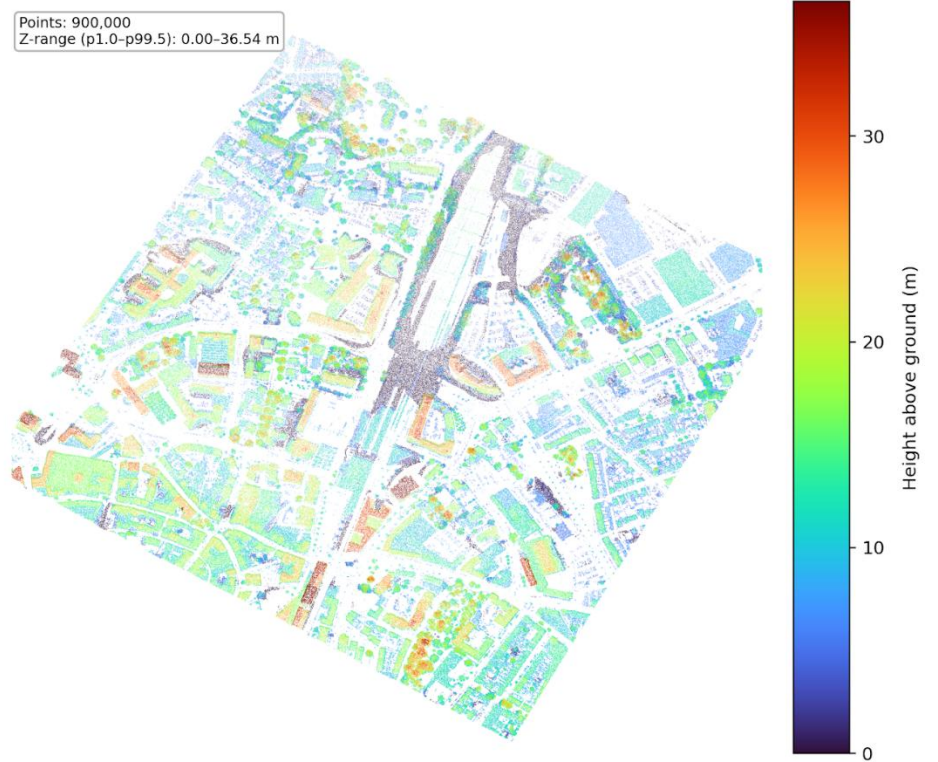
Table 4 Metrics comparison between full coverage (non-occluded) vs occluded samples .....51

# 1. INTRODUCTION

## 1.1. Background

Accurate and up-to-date 2D building outlines are fundamental to urban planning, cadastral mapping, infrastructure development and emergency response (Sulzer et al., 2025). In the past, building outlines were created manually or semi-automatically using satellite or aerial imagery but such traditional imagery-based building outline extraction methods often struggled with occlusions by trees, shadows and the diversity in roof geometries, hence leading to incomplete or incorrect 2d outlines in dense urban areas (Hattula et al., 2024).

Airborne Light Detection and Ranging (LiDAR) captures precise 3D measurements of the Earth’s surface and is processed into a Digital Surface Model (DSM) which encodes the elevation of all above-ground features. By subtracting the underlying ground model (Digital Terrain Model, or DTM) from the DSM, we obtain a normalized DSM (nDSM) that highlights only the height of objects above ground level (Oude Elberink et al., 2000). This nDSM representation makes buildings stand out clearly in complex urban environments, therefore greatly easing the task of footprint extraction even in areas with dense vegetation or closely spaced rooftops (Büyükdemircioğlu et al., 2022; Ji et al., 2024).



**Fig. 1** Point-based normalization of Point Cloud for a portion of Enschede, Netherlands

Although LiDAR-derived DSM or nDSM provides useful height information, accurately identifying building outlines from such products still requires advanced analysis techniques. Convolutional neural networks such as U-Net and related encoder-decoder models have shown strong performance for building

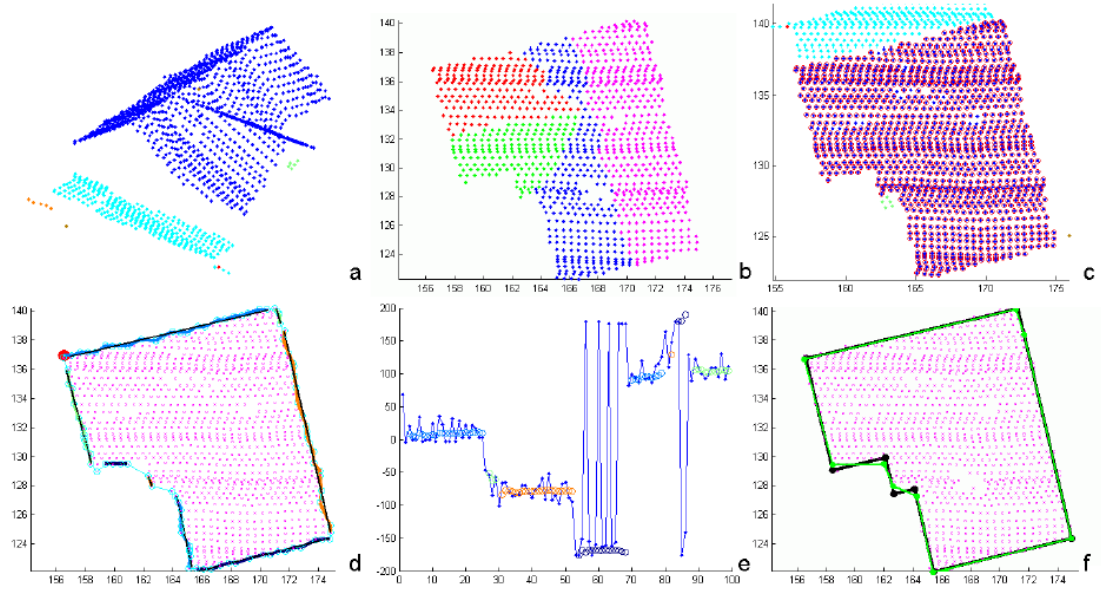
segmentation from nDSM or combined RGB-nDSM inputs, and multimodal combinations often improve robustness against spectral confusion or shadow-related errors (Buyukdemircioglu et al., 2022). However, these workflows still operate through an intermediate raster representation. As a result, sharp height discontinuities are often smoothed, and the predicted outputs usually take the form of masks that require later vectorization and geometric regularization to become suitable as clean building polygons. This is a well-recognized limitation in footprint extraction and regularization studies, where strong segmentation accuracy does not automatically translate into cartographically usable polygon geometry (Schuegraf et al., 2024).

Traditional LiDAR-based outline extraction methods, such as  $\alpha$ -shapes followed by generalization and regularization, remain important baselines for building-boundary generation. A representative example is Dorninger and Pfeifer (2008), where the outline is initialized from a 2D  $\alpha$ -shape and then generalized and regularized into straighter boundary segments. The limitation of such approaches is not that they are ineffective in general, but that they can become harder to apply consistently when outline conditions vary. Dorninger and Pfeifer explicitly note that the  $\alpha$  parameter must be chosen carefully, since large values may lead toward a convex hull and reduce concave detail, while the initial  $\alpha$ -shape may contain many short and irregular segments that require additional post-processing. Recent work confirms that  $\alpha$ -shape behaviour is highly sensitive to local point density and level of detail, so that a single fixed parameter is often not equally suitable for all buildings in a dataset (dos Santos et al., 2018).

For this reason, the present research does not aim to replace traditional methods in all situations. Instead, it has two aims. The first is to examine whether a graph-based learning framework can recover topologically valid, closed 2D building polygons directly from normalized airborne LiDAR point clouds, without the manually selected geometric parameters and post-hoc regularization that  $\alpha$ -shape pipelines require. The second is to characterize where such a learned approach performs well and where it degrades, across building outlines of increasing geometric complexity. Particular attention is given to concave buildings, moderately complex multi-corner buildings and shared-wall or row-house connected components, since these cases place the strongest demands on corner preservation, valid edge relations and final polygon closure and are precisely the conditions under which a fixed-parameter  $\alpha$ -shape may enclose the building area while failing to represent the intended outline.

To ground this comparison empirically, a fixed-parameter  $\alpha$ -shape baseline is evaluated on a representative subset of these difficult cases using the same vector-based metrics applied to the learned model, allowing a direct assessment of structural validity and parameter dependence on the building types where traditional methods are most challenged. The contribution of this thesis is therefore not a claim of universal superiority over established methods but evidence on whether a parameter-free, end-to-end graph-based formulation can produce structurally valid cadastral outlines, together with a clear account of its operating envelope and limitations under Dutch urban conditions.

An instance of how  $\alpha$ -shape workflows struggle in capturing the concave and convex transitions in building point clouds is shown in the figure below.



**Fig.2** Building outline determination initiated by mean shift segmentation (a) and (c) and planar face extraction (b). The points assigned to one building are shown as red circles in (c) and as magenta crosses in (d) and (f). The generalization of the 2D-shape (cyan polygon in (d)) is applied using an angular criterion (e). The generalized (green) and the regularized outline (black) are shown in (f). (Dorninger & Pfeifer, 2008)

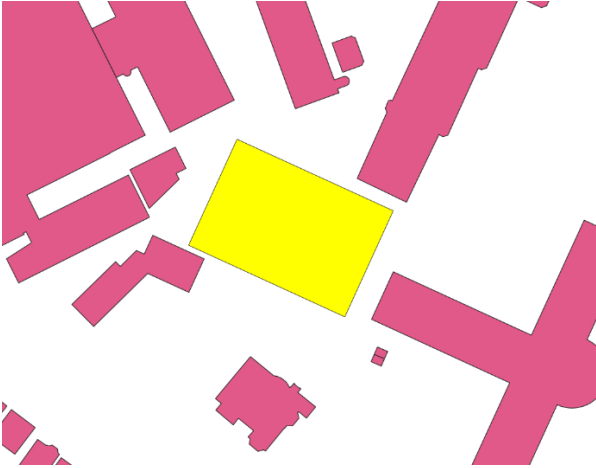
Recent research has tried to move closer to vector-aware prediction. Polygon-based frameworks such as PolyBuilding illustrate the value of predicting structured boundary geometry rather than only raster masks, although these methods are mainly developed for imagery rather than airborne LiDAR point clouds (Šanca et al., 2023). The motivation of the present research is therefore not to claim that traditional methods are ineffective but to investigate cases where they become difficult to tune or regularize consistently. In particular, this thesis investigates building point clouds whose outlines must be recovered as valid polygons under increasing corner complexity and spatial extent, including standalone poly-vertex buildings and large building point cloud instances. For such cases, a graph-based learning framework is favorable because it can learn boundary vertices and their valid edge relations directly from normalized airborne LiDAR point clouds instead of relying primarily on manually selected geometric parameters followed by later regularization.

In recent image-based end-to-end studies, the PolyR-CNN framework was introduced to extract building polygons from very high-resolution (VHR) RGB aerial imagery, relying on convolutional feature maps derived from 2D inputs (Jiao et al., 2024). However, PolyR-CNN is inherently limited to 2D imagery and lacks the capacity to directly process raw LiDAR point clouds. As a result, any attempt to adapt the model for LiDAR data would enable converting point clouds into rasterized representations such as DSM, nDSM or orthophotos, thereby introducing a pre-processing stage external to the network. This conversion can compromise boundary precision; for instance, interpolating sparse or uneven LiDAR returns into a DSM often produces discontinuous and blurred object edges (Mohamed et al., 2022). Furthermore, even if height information were incorporated (e.g., as an additional channel), it remains uncertain how effectively PolyR-CNN's architecture, originally designed for 3-channel RGB input, would integrate and utilize such geometric data.

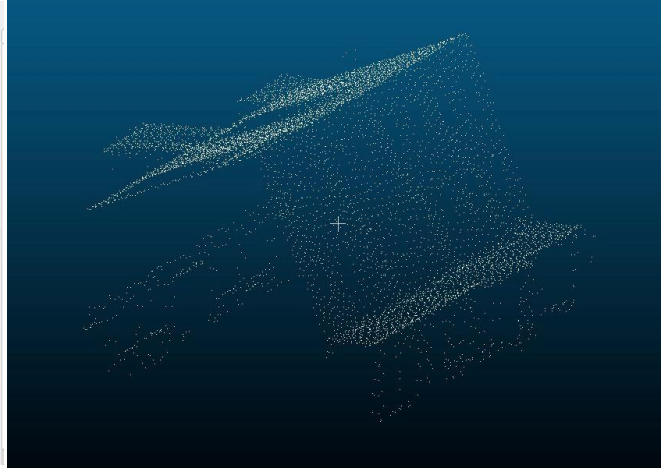
Direct point cloud methods preserve native 3D geometry and show promising progress toward edge and corner reasoning (Rottmann et al., 2022). Point2Roof introduced an end-to-end network that predicts roof

vertices and edges from airborne LiDAR via an autoregressive decoder, reducing the error accumulation typical of multi-stage model fitting; however, its target and evaluation are explicitly 3D roof-centric (Li et al., 2022). Point2Building extends autoregressive generation to full 3D meshes from pre-segmented building points, again prioritizing volumetric fidelity over planimetric polygon accuracy (Liu et al., 2024). Meanwhile, PBWR (CVPR 2024) and BWFormer push toward building wireframes from aerial LiDAR by regressing edges/corners with transformers, but they remain 3D-wireframe focused and do not deliver cadastral-grade 2D footprint polygons with vector-centric evaluation (Huang et al., 2024; Liu et al., 2025). Independent evidence that point clouds can yield high-fidelity outlines exists, for example, (Rottmann et al., 2022) extract 2D boundaries from 3D scans with high overlap to ground truth but the methods often assume curated scenes, pre-segmentation or substantial post-processing, leaving open the question of end-to-end footprint delineation.

The importance, both socially and scientifically, can be assessed such that it addresses the growing demand for accurate building data in expanding urban environments, while also contributing to the development of more advanced, efficient and GIS-compatible methods for building outline extraction. As cities continue to grow and digital cadastres become increasingly important, methods that make use of airborne LiDAR point clouds and deep learning will play a key role in shaping the future of urban geoinformation systems.



**Fig. 3.** Illustration of an example of BAG polygon in an urban setting of Enschede, Netherlands



**Fig. 4.** Illustration of an example building instance component point cloud

This research will be evaluated using airborne LiDAR datasets from the Netherlands' AHN (Actueel Hoogtebestand Nederland), for which point density typically ranges between 6 and 10 points/m<sup>2</sup> (Manders, 2023). LiDAR pipelines either operate in the nDSM/raster domain and optimize for mask accuracy, after which vectorization and regularization are needed where corner/edge quality is often degraded (Schuegraf et al., 2024) or operate in the point cloud domain but optimize for 3D roofs and meshes rather than cadastral quality 2D polygons (Li et al., 2022). What is missing is a method that directly learns 2D building outlines from normalized airborne LiDAR point clouds, targets vector regularity and topological validity in training and evaluation and is validated against authoritative BAG footprints under Dutch urban conditions (attached housing, narrow alleys and vegetation occlusion). By designing and developing a footprint-oriented deep learning model and grounding evaluation in vector-centric metrics (boundary Chamfer/Hausdorff, corner error, vertex count/angle regularity, topology), this work aims to bridge the gap between 3D focused point cloud learning and the practical cadastral need for clean and GIS-ready 2D building outlines on AHN data.

## 1.2. Problem Statement

Current deep learning approaches applied to airborne LiDAR focus largely on 3D roof or mesh reconstruction rather than on producing cadastral-quality 2D building footprints. For example, Point2Roof demonstrates autoregressive decoding of roof vertices and edges from point clouds (Li et al., 2022), while Point2Building generates 3D meshes from pre-segmented clouds without directly addressing planimetric polygon accuracy (Liu et al., 2024). As a result, many existing pipelines are evaluated primarily through raster-style metrics such as IoU and F1 which do not ensure regular corners, topological validity or cadastral usability. This limitation is also reflected in studies such as Rottmann et al. (2022), where raster-derived outlines are found to remain jagged and irregular.

This limitation becomes especially problematic in dense European urban areas, where attached row houses, narrow streets and vegetation occlusion further reduce the reliability of raster-based methods. A normalized point cloud, obtained by subtracting terrain elevation from the raw point cloud, provides per-point height-above-ground values that improve building separability while reducing terrain influence (Liu et al., 2017). A framework that operates directly on normalized point clouds can preserve geometric detail that is often weakened during rasterization and therefore provides a stronger basis for end-to-end 2D outline delineation. There is therefore a clear need to adapt point-cloud-based frameworks into models that can directly generate accurate, topologically valid 2D building polygons and evaluate them against authoritative BAG footprints using vector-centric metrics.

## 1.3. Research Gap

Recent advances show that deep networks can recover building geometry directly from airborne LiDAR point clouds but most of this work still targets 3D roof reconstruction or full 3D wireframes rather than cadastral-grade 2D footprints. Point2Roof (Li et al., 2022) demonstrates end-to-end prediction of roof vertices and edges from point clouds with an autoregressive decoder but its objective and evaluation are explicitly 3D roof-centric. Likewise, Point2Building (Liu et al., 2024) extends autoregressive decoding to full 3D building meshes from normalized building points and prioritizes surface realism rather than planimetric footprint fidelity. PBWR and BWFormer move toward wireframe primitives such as corners and edges but they also remain focused on 3D wireframe goals without a dedicated quantitative treatment of 2D polygon regularity, topological validity, and cadastral usability. Across these studies, end-to-end learning of 2D building outlines from raw LiDAR points, with the building polygon treated as the primary output, remains underexplored.

Moreover, authoritative Dutch datasets (AHN LiDAR, BAG footprints) are rarely used as the primary reference for rigorous planimetric evaluation. As a result, there is a lack of a clear, NL-specific assessment of how well direct point-cloud methods recover BAG-consistent 2D outlines at scale.

There is also a methodological gap between sophisticated 3D wireframe decoders such as PBWR, BWFormer and Point2Building and practical, reproducible pipelines aimed at delineating 2D building outlines. These 3D wireframe-oriented approaches often involve complex architectures and training procedures, including autoregressive decoding over many 3D tokens and substantial pre-processing or post-processing, which can make them computationally demanding and less straightforward to reproduce at scale. What is missing is a simplified end-to-end design with following key points:

- (i) A design that operates directly on normalized airborne LiDAR points
- (ii) A design that predicts 2D polygon vertices and edges with built-in cartographic priors (corner/straightness/topology).

(iii) A design that is benchmarked against BAG with vector-quality metrics, robustness and computational cost suitable for processing.

This thesis addresses these gaps by designing and developing a deep learning model that treats 2D building outline polygons as the primary output from normalized AHN point clouds. Particular attention is given to losses and decoding strategies that target vector regularity and topological validity, including corners, straight edges and closure, rather than only mask or mesh accuracy. BAG is used as the authoritative reference for evaluating the geometric and cadastral quality of the delineated outlines. By shifting the learning target from 3D roofs and wireframes to 2D cadastral outlines and by grounding evaluation in AHN and BAG, this work provides evidence on whether direct point-cloud inference can produce clean, GIS-ready building polygons.

## 2. OBJECTIVE AND RESEARCH QUESTIONS

### 2.1. Main Objective

To design, develop and evaluate a deep learning model that directly delineates 2D building outlines from airborne LiDAR point clouds and to assess its accuracy and cadastral usability by comparison with authoritative BAG building footprints in the Netherlands.

### 2.2. Subobjectives and Research Questions

**Sub-Objective 1:** Design and develop an architecture into an end-to-end framework capable of producing 2D building polygons directly from normalized point clouds.

**RQ 1.1:** How can the proposed deep learning model predict 2D footprint vertices and edges from normalized airborne LiDAR point clouds?

**RQ 1.2:** Which architectural modifications are most effective for achieving clean, topologically valid and cadastral-ready 2D outlines?

**Sub-Objective 2:** Evaluate the geometric and cadastral accuracy of the delineated 2D building outlines by comparing them against authoritative BAG building footprints.

**RQ 2.1:** How accurately do the delineated building outlines match BAG footprints in terms of geometric metrics such as Chamfer distance, Hausdorff distance and corner localization error?

**RQ 2.2:** To what extent do the predicted outlines meet cadastral-quality requirements including polygon regularity, topological correctness and alignment with legal building boundaries?

**Sub-Objective 3:** Examine the robustness of the proposed model in areas with varying levels of occlusion and complex urban morphology.

**RQ 3.1:** How does the model perform in less occluded environments (e.g., open residential areas) compared to moderately or heavily occluded urban scenes with trees and narrow streets?

**RQ 3.2:** To what extent do occlusion levels affect the model's ability to maintain geometric accuracy and polygon regularity and how does this performance compare to known limitations of LiDAR-based approaches in such settings?



## 3. MATERIALS AND METHODS

### 3.1. Datasets and Technologies

This research follows a structured workflow that converts airborne LiDAR point clouds into normalized building representations, prepares aligned training data using cadastral references and trains a deep learning network to delineate 2D building outlines directly from LiDAR-derived features. The complete workflow proceeds through six main stages consisting of data normalization, training-sample generation, model training and validation, inference, evaluation and comparison. A methodological workflow diagram can be seen in section 10, Annex A.

This research uses the Dutch national airborne LiDAR dataset (AHN), which provides dense point cloud coverage with an average density of approximately eight points per square meter and a vertical accuracy of 5-10 cm. The AHN dataset is particularly suitable for urban analysis in the Netherlands due to its nationwide availability and high positional accuracy (Actueel Hoogtebestand Nederland [AHN], 2021).

A Digital Terrain Model (DTM) is obtained from AHN data and a point-based normalized point cloud is then generated to emphasize buildings and vegetation while reducing terrain influence (Liu et al., 2017). Representative urban tiles are selected for this study, focusing on residential areas with row houses and narrow streets. The pre-segmented building point cloud components extracted from the normalized point cloud serve as the main input to the deep learning model.

As reference data, the Basisregistratie Adressen en Gebouwen (BAG), the official Dutch register of addresses and buildings, provides cadastral-quality 2D building polygons. This makes BAG a reliable ground reference for footprint delineation (Kadaster, 2023). BAG polygons are used both to create training labels and to evaluate the accuracy of predicted outlines, with spatial alignment ensured between AHN and BAG.

The implementation of this study relies on Python, a high-level programming language widely used for machine learning and deep learning applications. The core technology stack integrates tools for point cloud processing, deep learning, graph construction, and visualization. LAZ-format LiDAR files are processed for ground filtering and for generating normalized point clouds. The pyproj library is used to maintain spatial accuracy through coordinate transformations, particularly in the Dutch EPSG:28992 system.

PyTorch forms the backbone of the deep learning workflow. PointNet++ is used for point-wise feature extraction (Qi et al., 2017) and vertex-edge Wavefront OBJ files (.obj) are used for supervision in ordered polygon prediction. PointNet++ is well suited to this setting because it was developed for unordered point sets and learns local features hierarchically over local neighborhoods, which is beneficial for detecting corner-like and edge-like boundary structures (see Section 3.2.3 for more detail on the model architecture). Graph operations are managed through predicted vertices and their phase-based ordering, while polygon operations are handled by sorting the predicted vertex phases into a closed ring and converting the result into a GeoJSON polygon. The Shapely library is used for polygon creation and topology validation.

Finally, Matplotlib and QGIS are used for visualization and evaluation, allowing direct comparison of predicted building outlines against the cadastral reference dataset.

## 3.2. Methodology

### 3.2.1. Data Pre-processing

A Digital Terrain Model (DTM) is obtained from the AHN data portal and used to generate a normalized point cloud (nPC). Each point's elevation is replaced by its height above the local terrain, so that values represent height above ground rather than absolute elevation. This point-based normalization reduces the influence of terrain variation across the study area and makes elevated objects, such as buildings and vegetation, more distinguishable (Liu et al., 2017; Oude Elberink & Maas 2000). In the context of this research, normalization is important because the target task is 2D building outline delineation, and the geometric signal of the building should be made as prominent as possible before training.

After normalization, only the points already classified as buildings in the AHN dataset are retained. This filtering step isolates the building geometry from the surrounding terrain, vegetation and other non-building objects before connected components are generated. As a result, the model is trained on pre-filtered building components rather than on full mixed-scene LiDAR tiles. This is a deliberate methodological choice. The main objective of the study is to evaluate whether a deep learning network can learn to delineate building outlines from building point cloud components in a controlled graph-based setting. Under this setting, the model is trained and tested on a simplified and well-defined condition: the input is reduced to one building component, the target is one outline graph, irrelevant points are removed, and the model only needs to learn vertices, edges and polygon closure for that single component.

This pre-processing choice carries both advantages and limitations. On the one hand, removing non-building points simplifies the learning problem. The network no longer needs to separate building boundaries from surrounding terrain, roads, or vegetation and the spatial relation between the input component and its corresponding BAG outline becomes clearer. This reduces noise in the samples and makes the supervision more direct. On the other hand, excluding neighboring terrain also removes contextual information around the building boundary. In a tile-level setting, transitions between buildings and surrounding ground may provide additional spatial cues about the outer extent of the structure. These cues are not available in the present setup. The model is therefore trained and evaluated under a controlled component-based condition rather than under a raw tile-level condition.

Connected components are then extracted from the filtered building points. Each connected component is treated as a candidate building instance and later matched to a BAG polygon for sample generation. This step defines the basic prediction unit used throughout the research. Instead of learning from an entire tile containing many buildings, the model learns from one building component at a time. This makes the input structure consistent with the graph-based design of Point2Roof, which predicts one outline graph per sample.

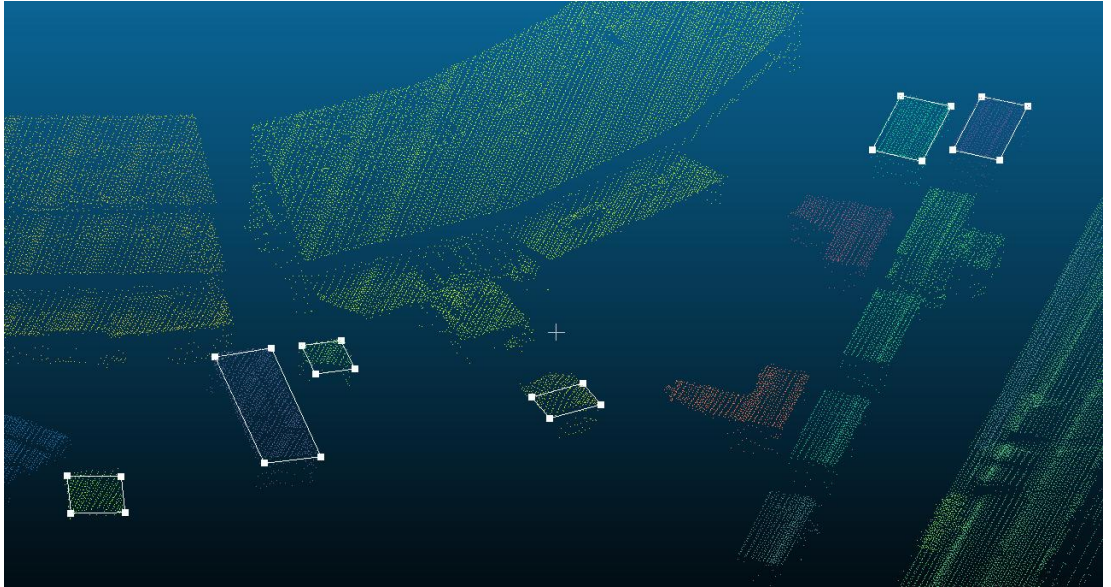
**Redundant Vertex Removal:** A further pre-processing step was applied to improve the geometric quality of the reference building outlines used for training. Because the model learns boundary vertices and edge relations directly from BAG-derived polygon annotations, the consistency of the reference geometry is highly important. In the raw BAG polygons, some outlines contained redundant vertices, for example multiple vertices located along a single straight edge without representing a true change in boundary direction. If such vertices are retained, they introduce unnecessary complexity into the supervision by increasing the number of target vertices and by creating edge relations that do not correspond to meaningful geometric corners. To address this, the reference polygons were cleaned before training by removing redundant vertices and preserving only the geometrically meaningful outline corners. As a result, the annotation files used for supervision better reflect the intended building structure and provide a cleaner correspondence between the point cloud component and the target graph.

This step is methodologically important because the adapted Point2Roof framework does not learn building area or mask overlap, but explicit graph structure in the form of vertices and edges. Even small inconsistencies in the reference polygon may therefore influence both corner learning and edge prediction. By simplifying the BAG outlines to their meaningful boundary vertices before converting them into graph annotations, the training data were made more consistent with the actual learning objective of the model. This also reduces the risk that failures during training or inference are caused by avoidable annotation irregularities rather than by the model behaviour itself.

The pre-processing strategy is therefore intended to produce geometrically clean and methodologically consistent training samples. Normalization emphasizes above-ground structure, building-point filtering restricts the task to the target object class, connected-components extraction transforms the data into units that are compatible with single-graph outline prediction, and reference-polygon cleaning improves the structural quality of the supervision. It should be noted, however, that this setup evaluates the model under pre-segmented building conditions and does not represent direct end-to-end delineation from raw airborne LiDAR tiles.

### 3.2.2. Training-Sample Label Generation

To train the model in a graph-based manner, the BAG building polygons must be converted into supervision that is compatible with point-cloud learning and graph prediction.

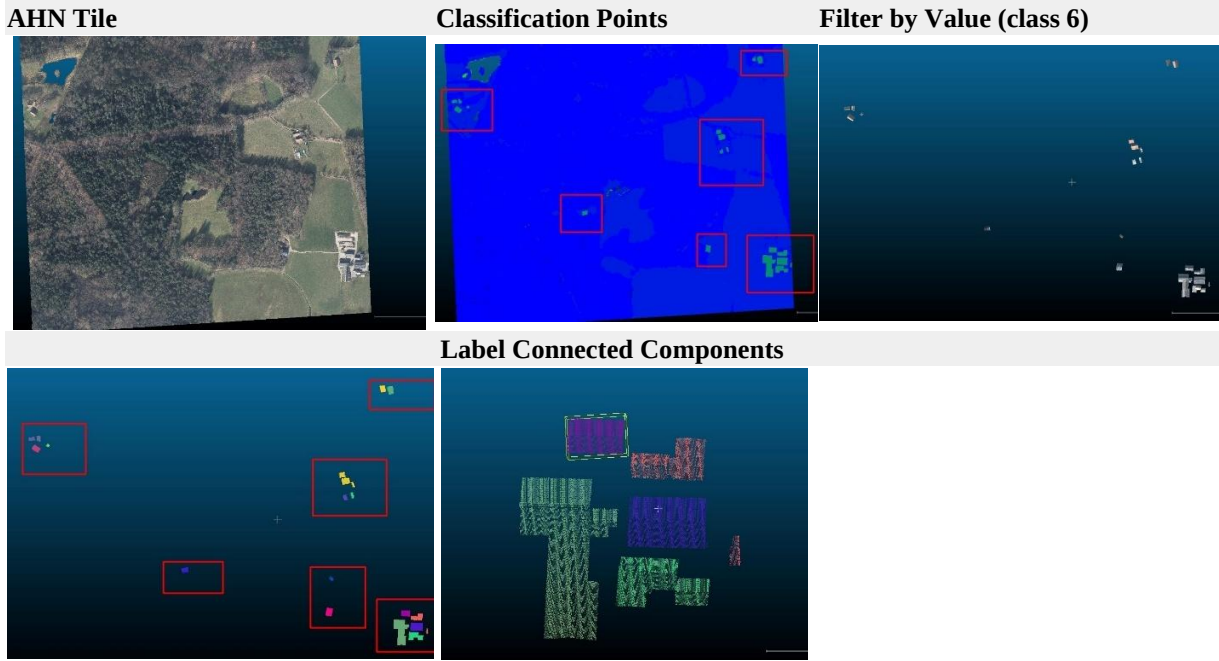


**Fig. 5** Illustration of BAG polygons (converted to object wavefront format) spatially mapped to corresponding components (standalone case)

In this research, training-sample label generation is therefore treated as a structured translation step between two data domains, that is, the normalized airborne LiDAR building component and the reference 2D building outline. The purpose of this step is not only to associate a BAG polygon with a point-cloud sample but also to convert that polygon into explicit learning targets for boundary vertices, offset regression and valid graph edges.

The process begins after connected components have been generated from the building-classified AHN points. Each connected component is treated as one candidate building sample. BAG polygons are then spatially matched to these components so that each selected point-cloud component is paired with one

corresponding building outline. In this way, the prediction unit used by the network, namely one connected building component becomes consistent with the supervision unit, namely one target outline graph.



**Fig. 6** Building instance components extraction from point clouds via connected components

After matching, each BAG polygon is converted into an object wavefront file, stored as `polygon.obj`. This file represents the building outline as an explicit graph. The polygon vertices are written as vertex records (v), while the boundary connectivity between successive vertices is written as line records (l). This conversion is methodologically important because the model does not learn from raster masks. Instead, it learns from a graph representation in which both the corner geometry and the edge adjacency are explicitly available. The resulting training sample therefore consists of a normalized point cloud (`points.xyz`) and a graph-encoded reference outline (`polygon.obj`).

```

1 v 5.213000 14.710000 2.458625
2 v 14.212000 4.858000 2.528972
3 v 9.167000 -0.107000 3.074081
4 v -0.227000 9.781000 3.015143
5 l 1 2
6 l 2 3
7 l 3 4
8 l 4 1

```

**Fig. 7** Annotation file (Object wavefront file)

```

points_n.xyz
polygon_n.obj

```

**Fig.8** Training Sample Pair Visual

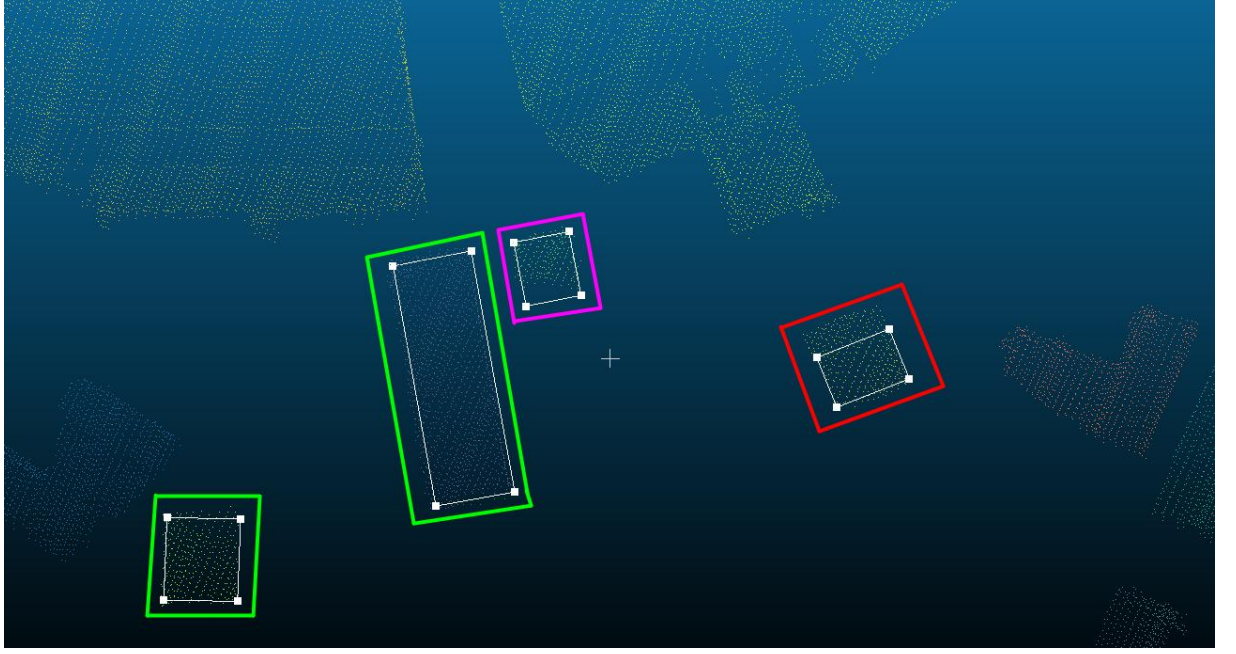
```

000001
000002
000003
000004
000005
000006
000007

```

**Fig. 9** Sequential structuring of each Training Sample Pair Visual

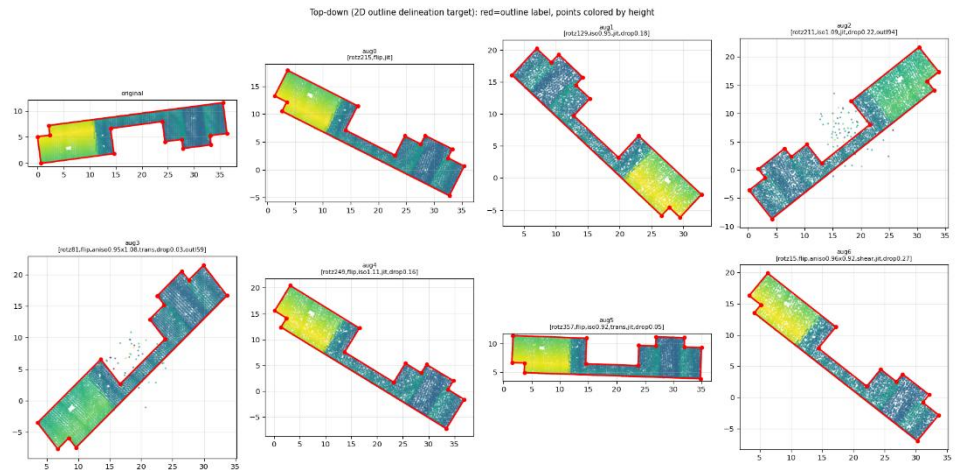
Components that are incomplete, ambiguous or temporally inconsistent with the BAG reference are excluded in order to reduce supervision errors and to maintain a reliable one-to-one relation between the input geometry and the target outline. Such exclusion of samples could also pose a risk of having too few representative and clean training samples for the deep learning network to learn valid outlines, so to address this problem data augmentation is done to compensate for excluded samples.



**Fig. 10** Accepted sample for training (Green). Excluded samples; (Pink) Imperfect overlap due to possible occlusion; Point cloud overshooting, imperfect overlap (Red)

Data augmentation was introduced as a compensatory strategy after quality-based sample exclusion. The removal of incomplete, ambiguous, or temporally inconsistent components improves the reliability of the supervision, but it may also reduce the number of representative training examples available to the network. This creates a trade-off between annotation quality and dataset diversity. For a graph-based deep learning model, this trade-off is particularly important because the network must learn not only general building presence but also corner structure, edge connectivity and polygon closure across different geometric conditions.

If the remaining dataset becomes too small or too narrow in its structural variation, the model may overfit to a limited set of building patterns and generalize poorly to unseen cases. Data augmentation is therefore used to compensate for excluded samples by increasing the effective size of the dataset and by exposing the model to a broader range of geometric configurations. This helps preserve learning diversity while still ensuring that the supervision is based on cleaner and more reliable reference outlines.



**Fig. 11** Illustration of a training sample augmentation

Point-wise vertex supervision is generated by comparing the input points with the set of reference polygon vertices. For each point in the normalized point cloud, a ball-query operation is used to determine whether the point lies within a fixed radius of a ground-truth vertex (Qi et al., 2017). If this condition is satisfied, the point is labelled as a positive corner-related point, otherwise, it is labelled as a negative non-corner point (binary classification). A positive label therefore does not mean that the point itself is the exact building corner. Rather, it means that the point belongs to the local neighborhood of a true/real corner and provides evidence for that corner. This formulation is appropriate for airborne LiDAR data, where a true building corner is usually supported by several nearby points rather than by a single exact observation.

For each positive point, an offset regression target is also created. This target is defined as the displacement from the point location to its assigned ground-truth vertex. The regression task therefore complements the binary classification task that is basically identifying and labeling points that belong to corner neighborhoods, while regression learns how these points should move toward the true vertex location. "In cases where local geometry could place a point near more than one vertex, the supervision must remain unique. Therefore, each point is assigned to one target vertex only so that the classification label and the offset regression target remain well defined" (Li et al., 2022).

Edge supervision is derived from the same explicit graph representation, but is formulated as a per-vertex ordering task rather than a per-pair adjacency task. Once the reference polygon has been converted into vertices and adjacency relations, the boundary defines a closed traversal in which each vertex has a well-defined position along the ring. Because the same polygon can be annotated starting from any vertex and in either direction, the reference traversal is first canonicalized to a deterministic convention counter-clockwise direction, with the lex-smallest vertex assigned position zero. After canonicalization, every reference vertex has a unique position in  $[0, n)$ , where  $n$  is the number of vertices in the polygon. This position is then mapped to a target angle on the unit circle,  $\varphi_{\text{target}} = 2\pi \cdot \text{position}/n$ , expressed as the two-dimensional vector  $(\cos \varphi_{\text{target}}, \sin \varphi_{\text{target}})$ . The unit-circle representation is used so that supervision is smooth and naturally handles wrap-around between the last and first vertex of the ring (Zhou et al., 2019).

During training, each predicted vertex is associated with a reference vertex through Hungarian matching, after which its target angle is taken from the canonical position of its matched reference vertex (Carion et al., 2020). Predictions that are not matched to any reference vertex receive no ordering supervision and are excluded from the loss; this prevents spurious detections from injecting contradictory targets into the network. The phase head is trained to minimize the angular deviation between its predicted  $(\cos \varphi_{\text{pred}}, \sin \varphi_{\text{pred}})$  and the target unit vector, using a cosine-similarity loss. At inference, the predicted angles are sorted to obtain a ring traversal and consecutive vertices in the sorted order are connected to form the boundary edges.

In this formulation, edge supervision is no longer based on pairwise adjacency decisions but on each vertex's position along the canonical boundary. The topological adjacency of the reference polygon is still preserved exactly because after sorting the predicted angles the consecutive pairs reproduce the same closed traversal that defines the reference edges.

This is important because two vertices that are close in space do not necessarily belong to the same boundary segment; the ordering view shifts the question from "are these two vertices connected" to "where on the boundary does each vertex lie," and edge connectivity emerges deterministically from the global ordering rather than from local pairwise decisions.



It is also important to underline that polygon consistency is not learned through a separate polygon-level supervision term. Instead, it is learned indirectly through the combination of correct corner localization and correct ring ordering. If the network recovers the relevant vertices and predicts a consistent phase for each, the sorted ring of consecutive vertices forms a closed polygon by construction. Polygon consistency is therefore represented in the training process through the structural correctness of vertex detection and per-vertex ordering rather than through a standalone raster or polygon-overlap target.

Overall, the complete label-generation strategy therefore provides two complementary forms of supervision. First, it provides geometric supervision at the point level through corner classification and offset regression. Second, it provides ordering supervision at the vertex level through canonical ring positions. This design is well aligned with the Point2Roof framework, since the model learns the boundary progressively, that is, from corner evidence in the point cloud, to candidate vertices and to their position along a closed traversal. As a result, the BAG reference data are transformed into a supervision structure that directly supports 2D building outline learning, where the outline emerges from sorting the predicted vertex positions rather than from explicit edge classification.

### 3.2.3. Model Architecture

The proposed model is an end-to-end deep learning framework that generates 2D building polygons directly from a segmented point cloud component, using vector supervision. It employs a PointNet++ architecture with an encoder-decoder structure to extract point-wise features from the 3D input point cloud (Qi et al., 2017). These features are used to classify each point as either a corner or a non-corner through binary classification. To refine the location of predicted corners, the model learns to regress offsets toward the ground-truth vertex positions provided in the vector annotation file. This regression helps localize compact groups of points around each corner (Li et al., 2022). The binary corner labels and offset targets used in this stage are generated from the reference polygon vertices through the ball-query assignment described in Section 3.2.2.

After this initial prediction stage, a clustering algorithm (DBSCAN) is applied to group nearby corner points, and the resulting cluster centroids are treated as the initial polygon vertices (Ester et al., 1996). Since these vertices may not align perfectly with the true building corners, their positions are further refined by a vertex refinement module which learns the residual displacement by comparing predicted vertices to the ground-truth annotations. The refined vertices form the candidate vertex set for boundary prediction. In contrast to the original Point2Roof framework (Li et al., 2022), the boundary is not obtained by independently classifying every candidate vertex pair as a valid or invalid edge. Instead, a phase prediction module assigns each refined vertex a position along the building boundary and edges are derived from this ordering.

**Phase Prediction Module:** In the original Point2Roof framework, the boundary graph is reconstructed by a Paired Point Attention module which independently classifies each candidate pair of detected vertices as a valid or invalid edge (Li et al., 2022). This approach treats edge prediction as a local pairwise decision and does not enforce that the resulting set of accepted edges forms a closed polygon, pointing out a critical architectural design gap.

In this research, the edge-prediction module is replaced with a phase prediction head that learns the position of each vertex along the canonical boundary traversal. Recovering a polygon by determining the order or connectivity of its vertices, rather than by classifying candidate edges independently, has precedent in polygonal building extraction: sequence-based methods such as Polygon-RNN (Castrejón et al., 2017) and PolyBuilding (Hu et al., 2023) generate the outline as an ordered sequence of vertices, while PolyWorld (Zorzi et al., 2022) formulates the connection of detected vertices as a global assignment

problem. The phase head adopts this ordering view in a deliberately simpler form: rather than an autoregressive sequence decoder or a pairwise assignment matrix, it regresses a single angular position per vertex and recovers the closed ring by sorting. The position is encoded as a unit vector  $(\cos \varphi, \sin \varphi)$  so that it varies continuously and without the wrap-around discontinuity of a raw angle (Zhou et al., 2019). This can be read as a learned counterpart to the classical practice of ordering polygon vertices by their angle about a centroid; because the predicted value encodes the learned traversal position rather than the geometric positioning of each vertex (Preparata & Shamos, 1985).

During inference, the predicted angles are sorted and consecutive vertices in the sorted order are connected to form the boundary edges. In this way, the network learns where each vertex lies along the boundary rather than which pairs of vertices should be connected and the resulting outline is a closed polygon by construction. The phase prediction module is structurally simpler than the original edge classifier. It produces one scalar output per vertex rather than one score per candidate vertex pair, which reduces the prediction problem from quadratic to linear in the number of vertices. It also removes the need for a separate closure or post-processing step, since sorted phases always produce a valid closed ring. The validity of replacing the pairwise edge classifier with a per-vertex ordering head therefore lies in the fact that boundary connectivity in a closed polygon is fully determined by the ordering of its vertices, so per-vertex ordering and pairwise adjacency are equivalent representations of the same topological information.

**Table 1** Comparison between Paired Point Attention Module and Phase Head

| Property                     | PPA module (Point2Roof original)                                              | Phase head                                                              |
|------------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <b>Prediction task</b>       | Per-pair binary classification: is this edge valid?                           | Per-vertex regression: what is this vertex's position along the ring?   |
| <b>Output dimensionality</b> | One scalar per candidate vertex pair (quadratic in vertex count)              | One 2D unit vector per vertex (linear in vertex count)                  |
| <b>Output representation</b> | Independent edge probabilities                                                | Predicted angular position on the unit circle                           |
| <b>Polygon closure</b>       | Not guaranteed; requires a separate cycle-detection step                      | Guaranteed by construction; sorting always produces a closed ring       |
| <b>Topological validity</b>  | Not guaranteed; may produce vertex degrees other than 2                       | Guaranteed by construction; every vertex has exactly two ring neighbors |
| <b>Loss formulation</b>      | Binary cross-entropy on edge labels                                           | Cosine-similarity loss on predicted phase angle                         |
| <b>Supervision unit</b>      | Edge pair $(n(n-1)/2)$ pairs per sample                                       | Vertex ( $n$ vertices per sample)                                       |
| <b>Failure modes</b>         | Disconnected edges, vertex-degree errors, open or self-intersecting rings     | Wrong ordering on vertices with similar local features                  |
| <b>Assumed output shape</b>  | Arbitrary graph (suited to 3D roof structures with hips, ridges, and valleys) | Single closed ring (suited to a 2D building footprint)                  |



The network architecture has three main parts:

1. A PointNet++ encoder-decoder that extracts per-point features, classifies corner points, and regresses offsets toward the nearest ground-truth vertex.
2. A clustering step using DBSCAN that groups predicted corner points into candidate vertices, followed by a refinement module that adjusts their positions toward the true building corners.
3. A phase prediction module that assigns each refined vertex a position along the canonical boundary, from which the closed polygon is obtained by sorting.

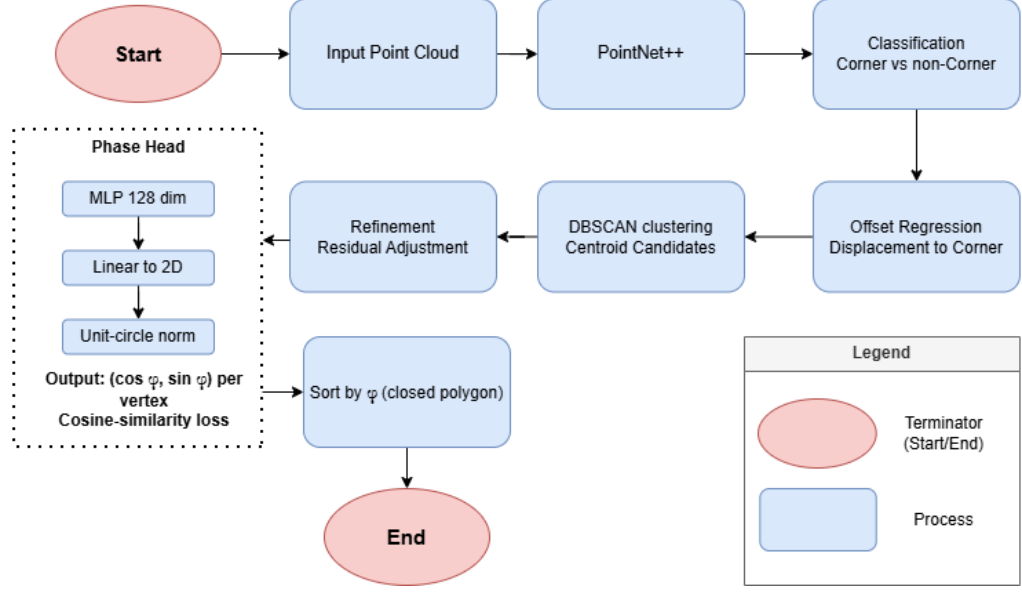


Fig. 12 Phase head module architecture

For vertex detection, the model employs two loss functions guided by the vector annotation file as ground truth. A focal loss (Lin et al., 2017) is used for the binary corner classification, and a Smooth-L1 loss is used for offset regression and vertex refinement (Girshick, 2015). For phase prediction, a cosine-similarity loss is used: the loss for each matched vertex is one minus the cosine of the angle between the predicted unit vector  $(\cos \varphi_{\text{pred}}, \sin \varphi_{\text{pred}})$  and the target unit vector  $(\cos \varphi_{\text{target}}, \sin \varphi_{\text{target}})$ . This loss is smooth across the full angular range and handles the wrap-around between the last and first vertex of the ring without discontinuity (Zhou et al., 2019).

**Methodological Reasoning for the Adopted Workflow:** The aim of this study is not only to detect buildings but to delineate clean 2D building polygons with meaningful vertices, valid boundary connectivity and closed outlines. A workflow was therefore required that treats the polygon as the primary output structure rather than as a later by-product of raster segmentation. A graph-based pipeline was adopted in which boundary evidence is learned at the point level, converted into candidate vertices, and then organized into a closed polygon through per-vertex ordering. This makes the workflow more consistent with cadastral requirements than approaches that first predict raster masks and then apply post hoc vectorization and regularization.

The workflow was further designed to operate directly on normalized airborne LiDAR point clouds rather than on rasterized products derived from LiDAR. This choice preserves the 3D geometric structure of the data and avoids the information loss that occurs during raster conversion. PointNet++ is well suited to this setting because it was developed for unordered point sets and learns local features hierarchically over metric neighborhoods, which is beneficial for detecting corner-like and edge-like boundary structures (Li

et al., 2022). Since building outlines depend on both local geometric detail and broader spatial context, point-based hierarchical feature learning provides a strong methodological basis for this task.

Point2Roof was selected as the starting framework because its internal prediction logic is already closely aligned with the requirements of 2D outline delineation. Although originally proposed for 3D roof graph reconstruction from airborne LiDAR, its core representation is a graph composed of predicted vertices and edges. This is structurally compatible with the present task, where the desired output is likewise a graph that must be converted into a valid 2D polygon. Compared with methods primarily designed for 3D mesh or wireframe reconstruction, Point2Roof provides a more direct basis for adapting airborne-LiDAR graph learning toward planimetric outline recovery. The main scientific contribution of this thesis is therefore not the development of an entirely new network, but the adaptation, supervision design and critical evaluation of an existing graph-based LiDAR framework for 2D cadastral-style building outline delineation.

The workflow was implemented on pre-segmented building components rather than on full LiDAR tiles. This was a deliberate methodological decision. Point2Roof is fundamentally a single-graph predictor. Training directly on tiles would combine instance separation and outline reconstruction into one problem, thereby increasing ambiguity in both vertex supervision and graph construction (Li et al., 2022). By working on connected building components, the prediction unit becomes consistent with the architectural design: one component corresponds to one target graph. This makes the learning problem more controlled, reduces graph ambiguity, and allows the model to focus on the central challenge of boundary reconstruction rather than on multi-instance separation.

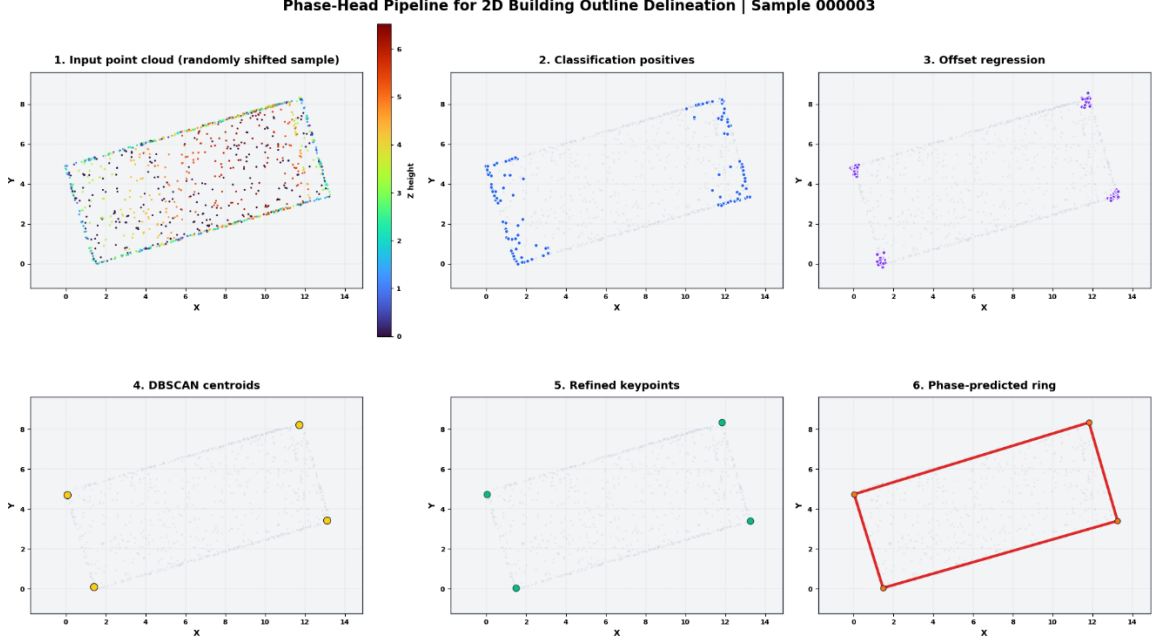
Within this framework, one targeted methodological change was introduced. The original Point2Roof edge classifier predicts boundary connectivity as independent decisions over candidate vertex pairs and offers no guarantee that the accepted edges form a closed polygon (Li et al., 2022). Since the present task explicitly requires closed building outlines, the edge classifier was replaced with a phase prediction head that learns each vertex's position along the canonical boundary traversal. Boundary edges are then obtained by sorting the predicted positions and connecting consecutive vertices. This change preserves the graph-based logic of Point2Roof while bringing the predicted topology into structural alignment with the target output, since a sorted ring is closed by construction.

The methodology was therefore designed from the structure of the desired output backward. Because the aim is to delineate closed 2D building polygons with meaningful vertices and valid boundary connectivity, a graph-based representation was preferred over raster prediction followed by vectorization. Point2Roof provided a logical starting framework because its prediction strategy is already based on vertex detection and graph reconstruction from airborne LiDAR (Li et al., 2022). In this study, the graph-learning logic was redirected from 3D roof reconstruction toward 2D outline delineation by replacing roof adjacency supervision with planimetric vertex ordering and by interpreting the final graph as a building boundary rather than a roof structure. Pre-segmented building components were used as the prediction unit so that the input representation, supervision design, model architecture and target output remain consistent with one another. The resulting workflow is considered the most methodologically logical choice for this research because it aligns data preparation, learning strategy and final output representation within one coherent graph-based framework.

#### **3.2.4. Detailed Working of Adapted Point2Roof Framework**

Point2Roof is a graph-based deep learning framework that predicts building outlines from a normalized airborne LiDAR point cloud (Li et al., 2022). In this study, the model is used for 2D building outline delineation from pre-segmented building components. The main idea is that the model first learns where

the important boundary vertices are likely to be, and then learns where each vertex lies along the closed building boundary. The full workflow combines differentiable learning components with non-differentiable geometric steps to convert point cloud input into a final polygon output.



**Fig. 13** Point2Roof 2D outline delineation adapted workflow illustration

**PointNet++ feature extraction:** The first stage of the model is based on PointNet++, which is used as the backbone for point-wise feature extraction. The input to the network is a normalized building point cloud where each point is represented by its three-dimensional coordinates, as introduced in Section 3.2.3. PointNet++ processes these points through a hierarchical encoder-decoder structure. In the set abstraction layers, local neighborhoods of points are sampled and grouped so that the network can learn geometric patterns at multiple scales (Qi et al., 2017).

In the feature propagation layers, these learned features are then passed back to the original point resolution. This design is important because building boundaries are defined by both local and global shape information. Local neighborhoods help the network learn corner-like and edge-like structures, while the hierarchical design helps the model understand the broader arrangement of the building component. As a result, PointNet++ produces a feature vector for every point, and these point-wise features form the basis for all subsequent prediction stages (Qi et al., 2017).

**Point-wise corner classification:** After feature extraction, the model performs point-wise binary classification. Each input point is classified as either a corner-related point or a non-corner point. This does not mean that each true building corner is represented by one single LiDAR point. Instead, the network learns which points lie within the local neighborhood of a true boundary vertex and therefore provide evidence for that corner. The output of this stage is a confidence score for every point, indicating the likelihood that the point belongs to a corner region. Corner detection is therefore formulated as a local evidence-learning problem rather than as a direct one-point-per-corner prediction task (Li et al., 2022). This stage is critical because all later graph construction depends on whether the model can first identify the important boundary regions. If too many true corner neighborhoods are missed, the final outline will be incomplete. If too many non-corner points are incorrectly classified as corner-related, the number of

candidate vertices increases unnecessarily and the later stages become more ambiguous. Reliable corner evidence is therefore one of the key requirements for successful outline reconstruction.

It is also useful to consider whether the network could additionally learn a point-wise edge classification task, in which points close to building boundary segments are labelled in addition to those near boundary corners. Such supervision could in principle provide useful information along long edges or in weakly supported boundary regions where corner evidence alone may be sparse. However, the adapted Point2Roof framework is primarily designed as a vertex-graph predictor rather than as a dense boundary-labelling model (Li et al., 2022). For this reason, the present implementation retains the original corner-first strategy in which the network first learns local corner evidence at the point level and boundary structure is recovered later at the graph level. A point-wise edge branch could be a meaningful future extension but it was not introduced here to keep the adaptation methodologically consistent with the original Point2Roof design.

Another important issue concerns the amount of point support needed for reliable corner detection. There is no single universal optimal number of points per corner, because corner detection depends on local point density, building size, corner geometry and the spatial distribution of points along the boundary. In this research, reliable corner detection is supported by restricting the input to pre-segmented building components, so that the available point budget is concentrated on the target structure rather than on terrain or neighboring objects. The fixed-point budget nevertheless remains a limitation, because larger or more complex buildings distribute the available point evidence more thinly across the boundary. Distinguishing corner types by geometric difficulty (for example, convex versus concave) and weighting them differently in the loss could be explored as a future extension, but was not adopted here to keep the loss formulation consistent with the standard Point2Roof training setup.

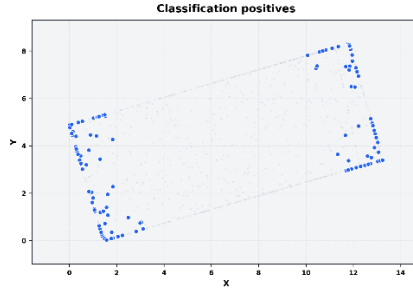
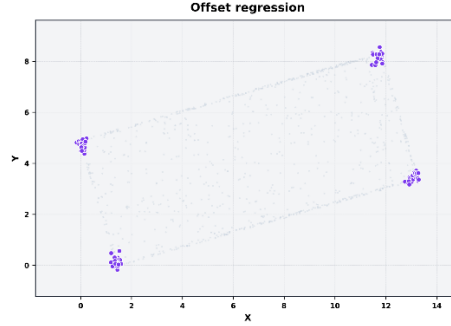


Fig. 14 Candidate corner points binary classification representation

**Offset regression:** In addition to classifying points as corner-related or non-corner, the model predicts an offset vector for each point. The purpose of this regression step is to move corner-related points toward the corresponding ground-truth vertex location, so that compact point groups can later be formed around meaningful boundary corners. In practice, the network learns the displacement from each point to its assigned target vertex. This reduces the spatial spread of the point-wise corner predictions and makes the later clustering step more stable. Classification and regression therefore work together: classification identifies the points that likely belong to corner neighborhoods, while regression sharpens their geometric position (Li et al., 2022).

The offset regression stage is important because raw point-wise predictions are often spatially scattered. Even correctly classified corner-related points may still be too dispersed to represent a usable graph vertex. By learning offsets, the network pulls these points toward a common geometric target, thereby improving the localization of candidate vertices before clustering (Li et al., 2022).

A methodological difficulty arises in short-edge regions or where two corners lie very close to one another. In such situations, a point may geometrically fall near more than one target vertex, but the regression target must remain unique during training. For this reason, each point is assigned to one target vertex only, through the point-to-vertex assignment step described in the label-generation procedure (Li et al., 2022). This ensures that the classification label and the offset regression target remain well defined. At the same time, very small edge lengths and closely spaced corner configurations remain challenging because the local supervision becomes more ambiguous, and the later clustering stage may still merge nearby corner evidence into one centroid (cluster absorption). This is one reason why very closely spaced concave-convex transitions are difficult cases for the model.



**Fig. 15** Candidate corner points offset regression representation

**DBSCAN clustering:** After the point-wise corner confidence scores and offset vectors have been predicted, the model applies DBSCAN as a non-differentiable clustering step. The purpose of this stage is to convert dense point-wise corner evidence into a smaller and more meaningful set of candidate corner groups, as introduced in Section 3.2.3. In practice, corner-related points that lie close to one another in space are grouped into clusters, while isolated predictions are treated as noise and are not used for later graph construction. This step is performed outside the differentiable learning pipeline, which means that gradients are not propagated through the clustering operation (Ester et al. 1996; Li et al., 2022).

This clustering stage is necessary because the model does not directly predict a fixed number of polygon vertices. Instead, it predicts local corner evidence at the point level. In airborne LiDAR data, a true building corner is rarely represented by one single exact point. More often, several nearby points provide partial support for the same geometric corner. DBSCAN is therefore used to identify compact groups of points that likely correspond to the same underlying boundary vertex. This makes the workflow flexible, since the number of corners may vary substantially from one building component to another (Ester et al. 1996; Li et al., 2022).

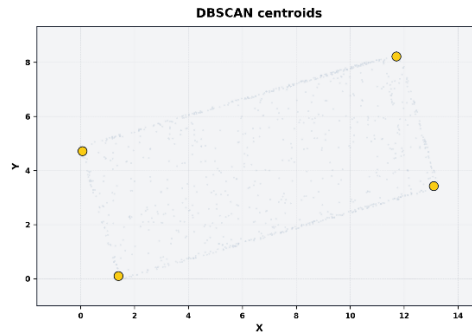
The suitability of DBSCAN in this framework lies in its ability to recover clusters without requiring the number of clusters to be known in advance (Ester et al. 1996). This is important for building-outline delineation because different buildings may contain very different numbers of corners. At the same time, the performance of clustering depends strongly on the quality of the earlier prediction stages. If corner confidence is weak, fragmented or spatially displaced or diffused, DBSCAN may fail to form the correct number of clusters. Similarly, in cases where nearby corners lie close together, the algorithm may merge them into one cluster, while weakly supported corners may disappear entirely (cluster absorption). For this reason, DBSCAN should not be viewed as an independent solution stage, but rather as a geometric aggregation step whose success depends directly on the strength and localization quality of the point-wise corner evidence (Li et al., 2022).

**Centroid generation:** After DBSCAN has grouped the corner-related points into clusters, then the centroid of each cluster is calculated and used as an initial estimate of a graph vertex. At this stage, the model moves from dense point-wise boundary evidence to a compact set of candidate polygon vertices. Each centroid represents the spatial center of one predicted corner cluster and therefore acts as a first approximation of a building-outline node. In this way, the workflow translates local geometric evidence into a graph-compatible representation (Ester et al. 1996; Li et al., 2022).

The centroid generation step is methodologically important because the final outline cannot be constructed directly from all corner-related points. A building boundary graph requires a manageable number of representative vertices rather than a dense collection of nearby point predictions. By replacing each cluster with one centroid, the model creates a reduced vertex set that can later be refined and connected through graph edges. This also makes the following graph-construction stage computationally feasible, since the number of candidate nodes is reduced substantially compared with the number of classified points.

At the same time, centroid generation is only an approximation of the true corner geometry. The centroid position depends on the spatial distribution of the clustered points and may therefore be influenced by uneven point support, offset errors or local asymmetry within the cluster. For well-formed corner clusters, the centroid can provide a good first estimate of the underlying boundary vertex. However, in more difficult situations, such as weakly supported corners or closely spaced corner transitions, the centroid may be shifted away from the true geometric corner or may represent a merged cluster that should actually correspond to more than one vertex. For this reason, centroid generation is a practical and necessary intermediate step but not the final solution for accurate vertex localization (Ester et al. 1996; Li et al., 2022).

The role of centroid generation in the full Point2Roof pipeline has two purposes. First, it transforms point-level corner predictions into discrete graph nodes. Second, it provides an initial geometric structure that can later be improved through vertex refinement. In this sense, centroid generation forms the bridge between local evidence learning and graph-based outline reconstruction (Li et al., 2022).



**Fig. 16** Centroid Generation via DBSCAN representation

**Vertex refinement:** Once the corner-related points have been clustered, the centroid of each cluster provides an initial estimate of a graph vertex. However, these centroids do not necessarily coincide exactly with the true building corners. The centroid location depends on the spatial spread of the clustered points and may therefore still be biased by uneven point support, offset inaccuracies or irregular local geometry. For this reason, Point2Roof includes a vertex refinement stage in which the initial candidate vertices are adjusted before graph construction. The goal of this step is to improve the geometric accuracy of the graph nodes so that they better represent the true boundary structure of the building (Li et al., 2022).

This refinement stage is especially important because errors in vertex position can propagate directly into the graph stage. Even a small displacement of one corner may later lead to incorrect edge confidence, missing connections or poor polygon closure. By refining the centroid positions, the model attempts to reduce these geometric errors before pairwise edge prediction begins. In this sense, the refinement stage acts as an intermediate correction step between local corner aggregation and graph topology learning (Li et al., 2022).

The stage is also important from a methodological perspective because centroid generation alone is only a geometric summarization, not a learned topological decision. A cluster centroid may lie near the true corner but not necessarily at the most informative graph position. The refinement module therefore uses learned geometric context to improve the candidate vertices beyond simple averaging (Li et al., 2022). At the same time, this stage has an important limitation of whether it can improve a coarse candidate vertex but it cannot recover a corner that was never detected in the earlier stages. If a true corner is completely missed during point-wise classification or absorbed into an incorrect cluster, the refinement step no longer has the information needed to reconstruct it fully (Li et al., 2022). For this reason, refinement improves vertex quality but it does not remove the dependence of the full pipeline on the earlier stages of corner evidence detection and clustering.

**Phase prediction:** After the candidate vertices have been refined, the model uses a phase prediction head to determine the order in which these vertices appear along the closed building boundary. Instead of evaluating every possible pair of vertices and deciding which pairs form valid edges, the network learns each vertex's position along the canonical boundary traversal. The boundary is then obtained by sorting the predicted positions and connecting consecutive vertices.

The phase head takes the refined vertex features as input and outputs, for each vertex, a two-dimensional unit vector  $(\cos \varphi, \sin \varphi)$  representing its angular position on a circle. The angle  $\varphi$  corresponds to where the vertex lies along the boundary, with all vertices distributed around  $360^\circ$  in canonical order. The network is trained to align each predicted unit vector with the target unit vector computed from the canonical position of the matched reference vertex, using a cosine-similarity loss (Preparata & Shamos, 1985; Zhou et al. 2019). During inference, the predicted angles are sorted to obtain a ring traversal and consecutive vertices in the sorted order are connected to form the boundary edges. Because the sorted ring always contains exactly  $n$  consecutive edges over “ $n$ ” vertices, the resulting polygon is closed by construction.

This stage is one of the most important parts of the framework because the final outline depends not only on detecting the correct vertices but on connecting them in the correct order. Even if all vertices are correctly localized, the outline will still fail if the phase head assigns an inconsistent ordering. The task itself is difficult because the network must learn to map a vertex's local geometric features to its global position along the boundary. This is a per-vertex regression problem rather than a pairwise classification problem, and its quality depends on the discriminative power of the refined vertex features.

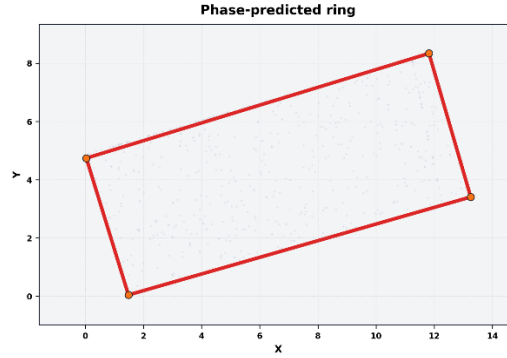
The phase formulation has two main advantages over the original pairwise edge classifier (Li et al., 2022). First, it scales linearly with the number of vertices, rather than quadratically with the number of candidate pairs (Zorzi et al., 2022), which keeps the prediction problem compact even for buildings with many corners. Second, closure of the predicted polygon is structurally guaranteed by sorting, rather than learned as an additional constraint.



The original Point2Roof edge classifier (Li et al., 2022) offered no such guarantee, since each pair was classified independently and the accepted edges did not necessarily form a closed ring. By replacing the pairwise classifier with a per-vertex ordering head, the adapted framework brings the predicted topology into direct alignment with the target output: a closed 2D building outline.

At the same time, the phase head has its own limitations. It assumes that each vertex has a distinguishable local geometric signature from which its canonical position can be inferred. For buildings with reflective or rotational symmetry, where several corners share nearly identical local context, the network may struggle to assign them to the correct canonical positions.

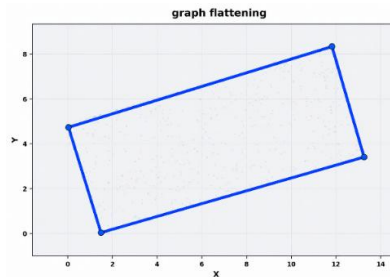
In such cases the predicted ring may be rotated relative to the reference, although its overall shape remains valid. The phase head also depends critically on the quality of the earlier stages. If a vertex is missed by classification, absorbed by clustering or poorly refined, no amount of ordering quality can recover the missing piece of the boundary.



**Fig. 17** Phase head’s output representation

**Graph flattening:** The predicted graph vertices may have varying Z values because the model processes the full 3D point cloud and the predicted vertices are influenced by the original point geometry. However, the objective of this research is the delineation of 2D building outlines. Therefore, a non-differentiable graph flattening step is introduced after graph prediction. In this step, the Z values of all predicted vertices are brought to a common value so that the graph becomes planar.

This flattening step is important for two reasons. First, it ensures that the final predicted outline is consistent with a 2D footprint representation rather than a 3D roof graph. Second, it simplifies subsequent polygon construction by removing unnecessary vertical variation between vertices. In practical terms, the graph flattening step converts the predicted graph from a three-dimensional boundary structure into a two-dimensional outline representation that is suitable for GIS-based polygon creation and evaluation.



**Fig. 18** Graph flattening representation



**Polygon extraction and GeoJSON creation:** After valid edges have been selected and the predicted graph has been flattened, the final step is polygon extraction. The graph is analysed to identify closed cycles that represent valid building outlines. Once a consistent closed boundary is reconstructed, the ordered vertex coordinates are written into a polygon structure. In this study, the final polygon is stored as a GeoJSON file (with EPSG: 28992 CRS), which allows direct use in geospatial analysis and comparison with BAG reference polygons.

This step is the final conversion from graph prediction to an explicit vector outline. It is also the stage at which graph errors become fully visible. If one or more important edges are missing, if the graph does not close or if the wrong vertices are connected, then a valid polygon cannot be extracted correctly. Therefore, polygon extraction is not only an output step but also an important validation of whether the preceding stages were successful.

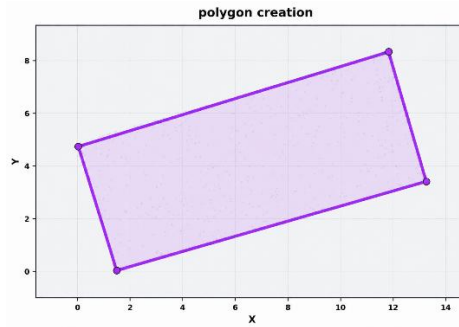


Fig. 19 Polygon creation representation

**Differential and non-differential components in the workflow:** The adapted Point2Roof framework combines both differentiable and non-differentiable components. The differentiable components include PointNet++ feature extraction, point-wise corner classification, offset regression, vertex refinement and phase prediction. These parts are learned directly from data through backpropagation. The non-differentiable components include DBSCAN clustering, centroid generation, phase sorting and final polygon extraction. These steps are not learned through gradient descent but they are necessary to convert the learned predictions into a usable building outline.

This combined design is methodologically important. The differentiable parts allow the model to learn geometric structure and per-vertex ordering from the training data, while the non-differentiable parts provide the deterministic geometric operations that translate these predictions into a final vector output. In this way, the model bridges the gap between raw point cloud learning and final polygon delineation.

**Workflow Significance:** For this study, the detailed working of Point2Roof is important because it shows how a roof-graph prediction framework can be adapted for 2D building outline delineation from airborne LiDAR point clouds. Rather than predicting raster masks or directly outputting polygons in one step, the model learns the problem as a sequence of geometric operations: detecting likely corners, refining their positions, predicting each vertex's position along the boundary, sorting the predicted positions and connecting them into a closed polygon. This makes the workflow especially suitable for research focused on boundary geometry, vertex structure and topological correctness.

At the same time, the workflow also explains why the model becomes more sensitive as shape complexity increases. Errors in corner detection, vertex refinement or phase prediction can propagate through the later stages and prevent correct polygon extraction. Understanding the detailed working of Point2Roof is

therefore essential not only for describing the method but also for interpreting its strengths and limitations in the later results chapters.

### 3.2.5. Configuration and Hyperparameters

The adapted Point2Roof framework has several configuration parameters that govern how its architectural stages operate. These parameters are not learned during training but are set externally and influence both the geometric assumptions of each stage and the connection between successive stages.

Before training and inference, each building component is rescaled by an isotropic min–max normalization applied per sample when the data are loaded. The minimum and maximum coordinates over all three axes are computed, every coordinate is translated by the global minimum and divided by the global coordinate range, and the same offset and scale are applied to  $x$ ,  $y$  and  $z$ . This places the component within the unit cube while preserving its aspect ratio, and the reference polygon is transformed by the identical scaling so that points and labels remain in the same frame. All spatial hyperparameters in Section 3.2.5 are therefore expressed in normalized units, that is, as a fraction of the component's coordinate range. The per-sample minimum and maximum are retained so that predicted vertices can be mapped back to metric coordinates (EPSG:28992) for the polygon output and visual comparison described in Section 3.2.7.

The values used in this study are summarised in the following, with each parameter explained in the paragraphs that follow.

**Table 2** Model configurations and parameters

| Parameter          | Value        | Units                                | Stage              |
|--------------------|--------------|--------------------------------------|--------------------|
| NPOINT             | 1024         | Points                               | Input Sampling     |
| PosRadius          | 0.15         | Normalized units                     | Vertex Supervision |
| ScoreThresh        | 0.5          | - (Dimensionless, score in $[0,1]$ ) | Vertex Filtering   |
| MatchRadius        | 0.2          | Normalized units                     | Hungarian Matching |
| Cluster eps        | 0.02         | Normalized units                     | DBSCAN clustering  |
| Cluster min_pts    | 4            | Points                               | DBSCAN clustering  |
| Refine SA Radii    | $[0.1, 0.2]$ | Normalized units                     | Refinement         |
| Refine SA Nsamples | $[16, 16]$   | Points                               | Refinement         |
| cls_weights        | 1.0          | - (Dimensionless)                    | Loss balance       |
| reg_weights        | 1.0          | - (Dimensionless)                    | Loss balance       |

All spatial parameters (PosRadius, MatchRadius, Cluster eps and the refinement set-abstraction radii) are expressed in normalized coordinate units, because each building component is rescaled to a canonical unit extent before being passed to the network rather than being retained in metres. Their metric equivalent therefore depends on the physical size of the building; for example, an eps of 0.02 corresponds to approximately 0.4 m on a building about 20 m across. NPOINT, Cluster min\_pts and the refinement sample counts are counts of points, while ScoreThresh, cls\_weights and reg\_weights are dimensionless.

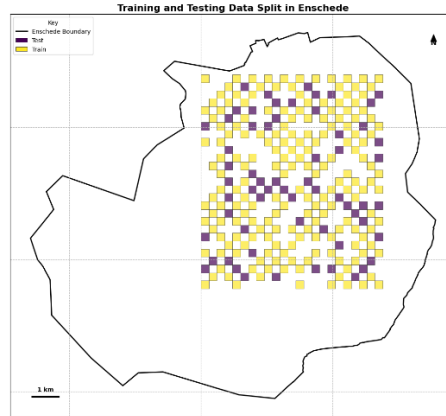
### 3.2.6. Model Training and Data Split Strategy

Training is conducted on GPU hardware using PyTorch. The model is trained on a split of 80:20 ratio, that is, 80 % for training and 20 % for testing. The training process is monitored using loss curves and evaluation metrics to ensure stability and reproducibility. A learning-rate scheduler gradually reduces the learning rate during training to support convergence while early stopping helps prevent overfitting and improves the model's ability to generalize across buildings with different shapes and roof structures (Li et al., 2022).

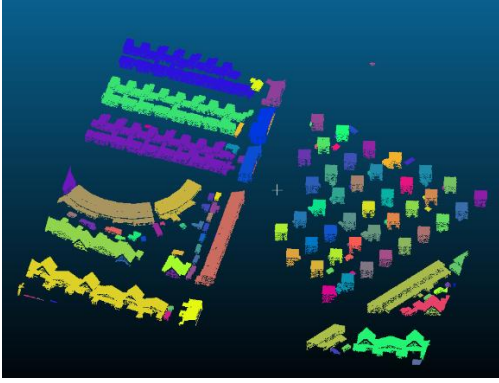
To ensure fair evaluation and reproducibility, the training and testing split used in this research follows the predefined spatial split provided by the Map2ImLas dataset (Anjanappa et al., 2026). Map2ImLas is a large-scale Dutch airborne benchmark containing 2,413 non-overlapping spatially matching tiles over approximately 217 km<sup>2</sup>, and it includes a non-overlapping train/validation/test partition for deep learning applications.

In this thesis, the same split was applied consistently to both the airborne point cloud tiles and the corresponding BAG polygon data for Enschede. As a result, training samples were generated only from training tiles, while testing samples were generated only from test tiles. This reduces spatial leakage and provides a stricter evaluation of model generalization, since buildings from nearby areas are not shared across training and testing.

This split strategy is also well aligned with the objective of the thesis. The aim is not only to show that the adapted Point2Roof framework can fit building outlines seen under similar local conditions, but also to assess whether the learned vertex-edge reasoning transfers to unseen urban blocks within Enschede. Using the Map2ImLas split therefore strengthens the validity of the evaluation, because success on the test set indicates that the model is learning generalizable outline patterns rather than memorizing local spatial structure.



**Fig. 20** Training and Testing Data Split for Enschede Municipality



**Fig. 21** AHN 6 Tile (connected components)  
Spatially clipped to MapLmLas Tile extent



**Fig. 22** BAG polygons overlaid over MapLmLas training  
& testing tiles (**Blue:** training & **Red:** testing)

### 3.2.7. Model Inference and Polygon Generation

During inference, the trained model takes a pre-segmented and normalized point cloud as input and predicts corner probabilities, offset vectors and per-vertex phase positions using the learned PointNet++ backbone and task heads. Predicted corner-related points are first shifted by their predicted offsets and then grouped using DBSCAN, and the centroids of the resulting clusters are treated as the initial polygon vertices. These initial vertices are passed through the refinement head, which produces a small residual adjustment that improves their alignment with the true corner positions. The phase prediction head then assigns each refined vertex a position along the canonical boundary, represented as a two-dimensional ( $\cos \varphi$ ,  $\sin \varphi$ ) unit vector (Ester et al., 1996; Li et al., 2022; Qi et al., 2017; Zhou et al., 2019).

To obtain the final outline, the predicted angles are sorted and consecutive vertices in the sorted order are connected, which produces a closed Hamiltonian ring by construction. The output is therefore a closed building polygon without a separate cycle-detection or closure-enforcement step. Optional post-processing such as polygon simplification, for example, Ramer-Douglas-Peucker (Douglas & Peucker, 1973) can be applied to smooth the final output while preserving geometric fidelity.

### 3.2.8. Evaluation Metrics

Model performance is assessed using both vector-based geometric metrics and topological validity indicators. The vector metrics include:

- Chamfer distance and Hausdorff distance between predicted and BAG boundaries.
- Precision, Recall, f1-score and IoU
- Vertex precision and recall within a relevant tolerance value.
- Area and centroid displacement compared to BAG.

Topological metrics assess polygon validity, closure and absence of edge crossings. This combination ensures that evaluation reflects cadastral-level geometric quality rather than only raster overlap, as with IoU (Fan et al., 2017; Huttenlocher et al., 1993).

### 3.2.9. Comparative Evaluation with Poly R-CNN

To assess the benefits of point-cloud-based learning, results from Point2Roof are compared with Poly R-CNN (Jiao et al., 2024), an image-based end-to-end polygon prediction model that operates on very high-resolution (VHR) RGB orthophotos and directly predicts 2D building footprints as ordered sequences of polygon vertices.

The comparison evaluates geometric accuracy (Chamfer/Hausdorff), vector regularity (angle and edge length deviation) and computational efficiency. Through this comparison, the study quantifies whether or not LiDAR point processing yields more accurate and cadastral-ready 2D outlines than raster-based methods.

## 4. RESULTS AND DISCUSSION

### 4.1. Training Details

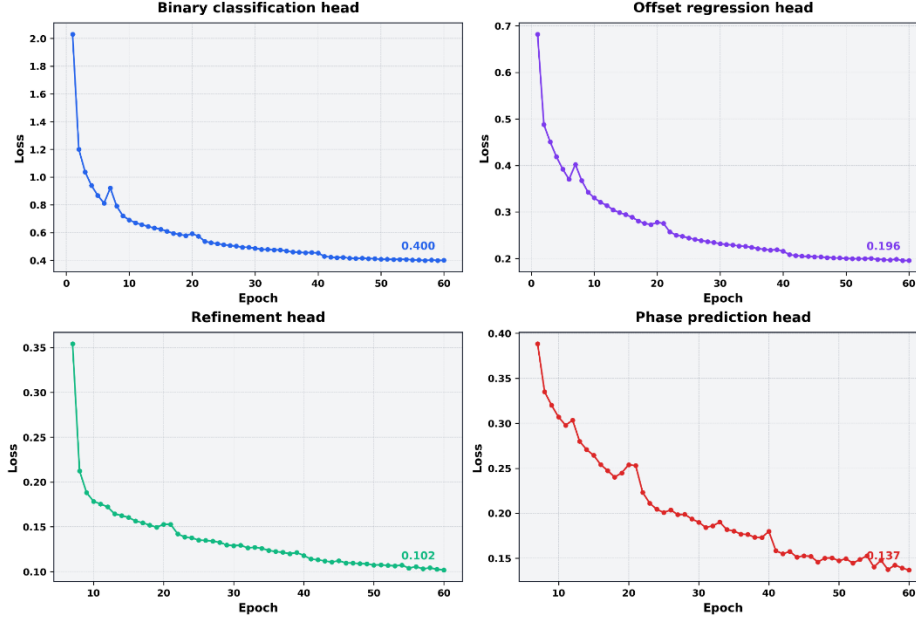
The network is trained end-to-end using the Adam optimizer (Kingma and Ba, 2014) with an initial learning rate of  $1 \times 10^{-3}$  and weight decay of  $1 \times 10^{-3}$ . A step learning rate schedule is applied, in which the learning rate is multiplied by 0.5 every 20 epochs. This schedule allows the network to make large early updates while the losses are still descending steeply and to refine its predictions with smaller updates as training stabilizes. The batch size is set to 64 and the network is trained for 60 epochs in total. Each input point cloud is uniformly sampled to 1024 points before being passed to the network, which keeps the input size constant across building components of different sizes and ensures that the per-batch memory cost remains predictable (Li et al., 2022).

The training proceeds in two phases within a single optimization run. For the first 5 epochs, only the per-point vertex detection heads are active. During this warm-up phase, the network learns the binary classification of corner-related points and the offset regression toward ground-truth vertex positions, without yet propagating gradients through the downstream clustering and ordering stages (Li et al., 2022). This is motivated by the observation that the later stages depend on stable vertex predictions, so allowing the vertex heads to converge to a reasonable initial state before downstream supervision is added improves the stability of joint training. From epoch 6 onwards, the cluster refinement and phase prediction heads are activated, and all four loss components are optimized jointly until the end of training. The network is trained on the CRIB Platform (University of Twente) using a single GPU.

All loss components are logged per epoch and saved to a comma-separated value file alongside the model checkpoints. This logging makes it possible to inspect the convergence behaviour of each head independently after training, which is particularly informative in a multi-head setting where individual losses can have different scales and convergence rates.

### 4.2. Training Convergence

The following figure shows the per-epoch training loss for each of the four heads over the 60-epoch training run. Several observations can be made from these curves.



**Fig. 23** Training losses curves for binary classification, offset regression, refinement and phase prediction heads

The classification and offset regression losses descend monotonically from epoch 1 onwards, starting at 2.03 and 0.68 respectively and reaching 0.40 and 0.20 at the end of training. This represents a reduction of approximately 80 % for the classification loss and 71 % for the offset regression loss. The classification loss continues to descend slowly throughout training, while the offset loss begins to plateau around epoch 40, which suggests that the spatial localization of corner-related points reaches its noise floor earlier than the binary discrimination problem. The discrete step changes in the slope of both curves at epochs 20 and 40 correspond to the learning rate reductions imposed by the scheduler.

The refinement loss only begins to be logged from epoch 7 onwards, since the cluster refinement head is inactive during the 5-epoch warm-up phase. From its first logged value of 0.35, it descends rapidly during the first ten epochs of joint training and then settles into a slower descent, reaching 0.10 by epoch 60. The fast initial descent reflects the fact that the cluster centroids produced by DBSCAN already provide a reasonable spatial estimate of the vertex location, so the refinement head only needs to learn small residual corrections. Once these systematic residuals are absorbed, further improvements come more slowly.

The phase prediction loss also begins at epoch 7 and shows the longest descent trajectory of all four heads. It drops from approximately 0.39 to 0.14 by epoch 60, with the steepest slope occurring between epochs 20 and 50. Because the phase loss is a cosine-similarity loss whose values map directly to angular error, the final value of 0.14 corresponds to an average angular deviation of approximately 31 degrees between predicted and target ring positions. For a polygon with up to 12 vertices, where the canonical angular spacing between adjacent vertices is 30 degrees, this places the model just within the regime in which the predicted sort order recovers the true boundary traversal on most samples. The phase loss is still descending at epoch 60, which indicates that the head has not fully converged and that longer training could yield further improvement.

The total loss, which is the unweighted sum of the four head losses, descends from 2.71 at epoch 1 to 0.83 at epoch 60. The fact that all four head losses descend simultaneously confirms that the joint optimization does not produce destructive interference between the heads.

### 4.3. Quantitative Analysis

This section reports the numerical evaluation of the trained model on the held-out test set and uses the results to answer research questions 2.1 (geometric accuracy of the predicted outlines) and 2.2 (cadastral-quality requirements). The evaluation is structured in three parts: an overview of the metrics and the test set; the geometric accuracy analysis that addresses RQ 2.1; and the cadastral-quality analysis that addresses RQ 2.2.

#### 4.3.1. Evaluation Setup and Metric Summary

The trained model was evaluated on 1,029 held-out building components drawn from the same data distribution as the training samples. Of these, 1008 produced a valid predicted polygon, 21 failed to produce non-self intersecting polygons (typically due to error propagation reaching the phase prediction stage). The reported metrics therefore cover 1,029 samples, with failures included as zero-overlap predictions in the area-based statistics and excluded only from the geometric-distance metrics where they would otherwise be undefined.

Three evaluation categories of metrics are used. Area-based metrics (precision, recall, F1 score, IoU) measure how well the predicted polygon overlaps the BAG reference polygon as a planar region. Shape and boundary metrics (C-IoU, Boundary IoU) add sensitivity to where the overlap occurs: C-IoU penalizes vertex-count mismatch in addition to area difference and Boundary IoU computes overlap only in a thin band around the polygon boundary, isolating boundary-localized errors from interior alignment. Geometric distance metrics (PoLiS, Chamfer distance, Hausdorff distance) measure how close the predicted boundary lies to the reference boundary in metric terms. Additional diagnostic metrics report the ratio of predicted to reference vertex count (N-ratio), the fraction of samples for which the predicted and reference vertex counts match exactly, and the count of geometrically invalid (self-intersecting) predictions.

**Performance Under Configuration:** The trained model was evaluated on the held-out test set using the configuration  $\text{min\_pts} = 4$  and  $\text{eps} = 0.02$ . The following table shows the run values.

**Table 3** Quantitative evaluation of inferred samples

| Metric                             | Value |
|------------------------------------|-------|
| <b>Area Based (RQ 2.1)</b>         |       |
| Precision                          | 0.901 |
| Recall                             | 0.884 |
| F1 Score                           | 0.888 |
| IoU                                | 0.862 |
| <b>Shape and boundary (RQ 2.1)</b> |       |
| C-IoU                              | 0.784 |
| Boundary IoU                       | 0.783 |
| <b>Geometric distance (RQ 2.1)</b> |       |
| Mean PoLiS                         | 0.180 |

|                                                |                |
|------------------------------------------------|----------------|
| Mean Chamfer                                   | 0.177          |
| Mean Hausdorff                                 | 0.561          |
| Mean N-ratio                                   | 0.830          |
| <b>Topology and cadastral quality (RQ 2.2)</b> |                |
| Mean N-ratio                                   | 0.835          |
| Vertex-count recall                            | 0.804          |
| Exact corner-count match                       | 35.7%          |
| Self-intersecting predictions                  | 21/1029        |
| Prediction failure rate                        | 21/1029 (2.0%) |

#### 4.3.2. Geometric Accuracy (RQ2.1)

RQ 2.1 asks how accurately the predicted building outlines match BAG footprints in terms of geometric metrics such as Chamfer distance, Hausdorff distance and corner localization error. The numerical results that depend on this question are the area-based metrics, the shape and boundary metrics and the geometric distance metrics.

The model achieves an F1 score of 0.888 and an IoU of 0.862 on the evaluated test set, indicating that the predicted polygons capture the location and approximate extent of the building footprint consistently across the 1,029 evaluated samples. The precision-recall balance is slightly weighted toward precision (0.901 versus 0.884), meaning that when the model declares a region as building footprint, it does so with high confidence, while a small fraction of true building area is missed. This is the expected pattern given the architecture's design: the classification head's score threshold and DBSCAN's `min_pts` filter both bias the pipeline toward fewer but more reliable predictions.

The Boundary IoU of 0.783 is recorded lower than the unadjusted IoU of 0.862 which is a gap of approximately eight percentage points. This gap is the most diagnostic finding for RQ 2.1 because Boundary IoU isolates the region of the polygon where geometric accuracy actually matters, that is, the boundary. The interpretation is that the polygon interior is recovered correctly in the vast majority of cases (driving the high IoU), but the boundary itself drifts more from the reference than the area metric alone suggests. This is consistent with how the architecture works, which is, small displacements of individual vertices have only a small effect on the interior area but a proportionally larger effect on the boundary band that Boundary IoU measures.

The geometric distance metrics quantify this boundary displacement in physical terms. The mean PoLiS distance of 0.180 in normalized coordinates indicates that the average vertex of a predicted polygon lies approximately 0.18 normalized units from the nearest edge of the reference polygon. The mean Chamfer distance of 0.177 confirms that this displacement is symmetric, predicted boundary points and reference boundary points are on average a similar distance from each other in both directions. The mean Hausdorff distance of 0.561, however, is moderately larger. It reports the worst-case point-to-point distance, averaged across samples and indicates that the worst-aligned boundary point on a typical predicted polygon lies almost one normalized unit from the reference. The ratio of Hausdorff to Chamfer is approximately 3.1, which characterizes the failure-mode shape: most of the boundary aligns closely with



the reference but each polygon contains one or two points that are markedly further off typically the corners that were missed or absorbed by the clustering stage, which produce a "cut corner" detour in the predicted boundary that the worst-case metric captures.

Together, these three metric families show that the model produces geometrically accurate outlines in the majority of cases, with average boundary displacement small relative to building dimensions but with localized worst-case errors that drag the Hausdorff metric up. The answer to RQ 2.1 is therefore that the predicted outlines match BAG footprints closely on average, which is, IoU 0.862, Boundary IoU 0.783, mean Chamfer 0.177 with the principal limitation being localized failures at individual corners rather than systematic shape distortion across the polygon.

#### **4.3.3. Cadastral Quality Requirements (RQ2.2)**

RQ 2.2 asks the extent to which the predicted outlines meet cadastral-quality requirements, including polygon regularity, topological correctness and alignment with legal building boundaries. These properties are categorical rather than continuous and require separate treatment from the geometric accuracy assessment.

Regarding topological correctness, out of the 1008 predicted polygons produced by the model, 21 are self-intersecting. This is a direct consequence of the phase head architecture described in Section 3.2; however the polygon is constructed by sorting predicted phase angles and connecting consecutive vertices, which guarantees by construction that the resulting ring is a simple closed polygon with each vertex incident to exactly two edges. The original Point2Roof edge classifier offered no such guarantee its output could in principle contain edges that crossed or vertices with the wrong degree and required a separate cycle-detection step to enforce closure. The phase head eliminates this entire class of failure modes at the architectural level. For cadastral applications, where topologically invalid polygons cannot be ingested into a registration system without manual correction, this property is particularly valuable.

Polygon regularity and vertex count are equally significant. The mean N-ratio of 0.835 indicates that the predicted polygons contain on average approximately 84% of the reference vertex count, equivalent to a systematic under-prediction of about 16% of corners. The vertex-count recall of 0.804 measures the same property from a different angle and is consistent with this finding. Only 35.7% of samples produce a prediction with exactly the same number of vertices as the reference, meaning that for the majority of test samples, the topological detail of the predicted polygon differs from the cadastral reference, even when the overall shape is correctly recovered. This under-prediction is the principal limitation of the current pipeline and originates in the architectural mechanism described in Section 3.2.4: weak-support corners that fall below the classification threshold or are absorbed by adjacent clusters during DBSCAN do not reach the refinement and phase prediction heads and the resulting polygon "cuts the corner" with a straight edge between the surrounding detected vertices. The C-IoU of 0.784, approximately eight percentage points below the unadjusted IoU, directly quantifies the cost of this under-prediction in cadastral terms.

Alignment with legal building boundaries directly addresses the BAG reference polygons used as ground truth are themselves the legal cadastral boundaries used in the Dutch building registration system. The IoU of 0.862, F1 of 0.888 and Boundary IoU of 0.783 reported in Section 4.3.2 therefore directly measure alignment with legal boundaries. Within the 1008 successful predictions, this alignment is sufficient for visual comparison and approximate boundary recovery but does not yet reach the precision required for direct ingestion into the cadastral registration system which typically demands sub-centimeter to centimeter-scale boundary precision. The current results are consistent with a pipeline suitable for outline proposal and downstream review rather than for fully automated cadastral entry.

The answer to RQ 2.2 is therefore that the predicted outlines meet some cadastral-quality requirements unconditionally (topological correctness is guaranteed by the phase head architecture, with the outputs in the test set), they fall short on the vertex-count requirement (35.7% of samples reproduce the exact reference vertex count) and they meet the alignment requirement at a level appropriate for human-supervised review but not for fully automated cadastral entry. The principal direction for closing these gaps is improving the recall of weakly-supported corners, which would directly raise the N-ratio and the exact-match rate without compromising the topological guarantees already provided by the phase head.

#### **4.3.4. Summary**

The quantitative evaluation answers the two research questions as follows. For RQ 2.1, the predicted outlines match BAG footprints with an average IoU of 0.862 and a mean Chamfer distance of 0.177; the principal geometric limitation is localized worst-case error at individual corners rather than systematic shape distortion. For RQ 2.2, the predicted outlines are topologically consistent by construction (out of 1008 successful predictions) and align with legal building boundaries at a precision suitable for human-supervised cadastral proposal but not for fully automated entry; the vertex-count under-prediction at only 16% on average is the principal cadastral-quality limitation. The findings collectively indicate that the adapted Point2Roof framework with the phase head produces reliable polygon outlines for the typical case, with limitations that are well-localized architecturally and that suggest specific directions for future work.

#### **4.4. Success Cases**

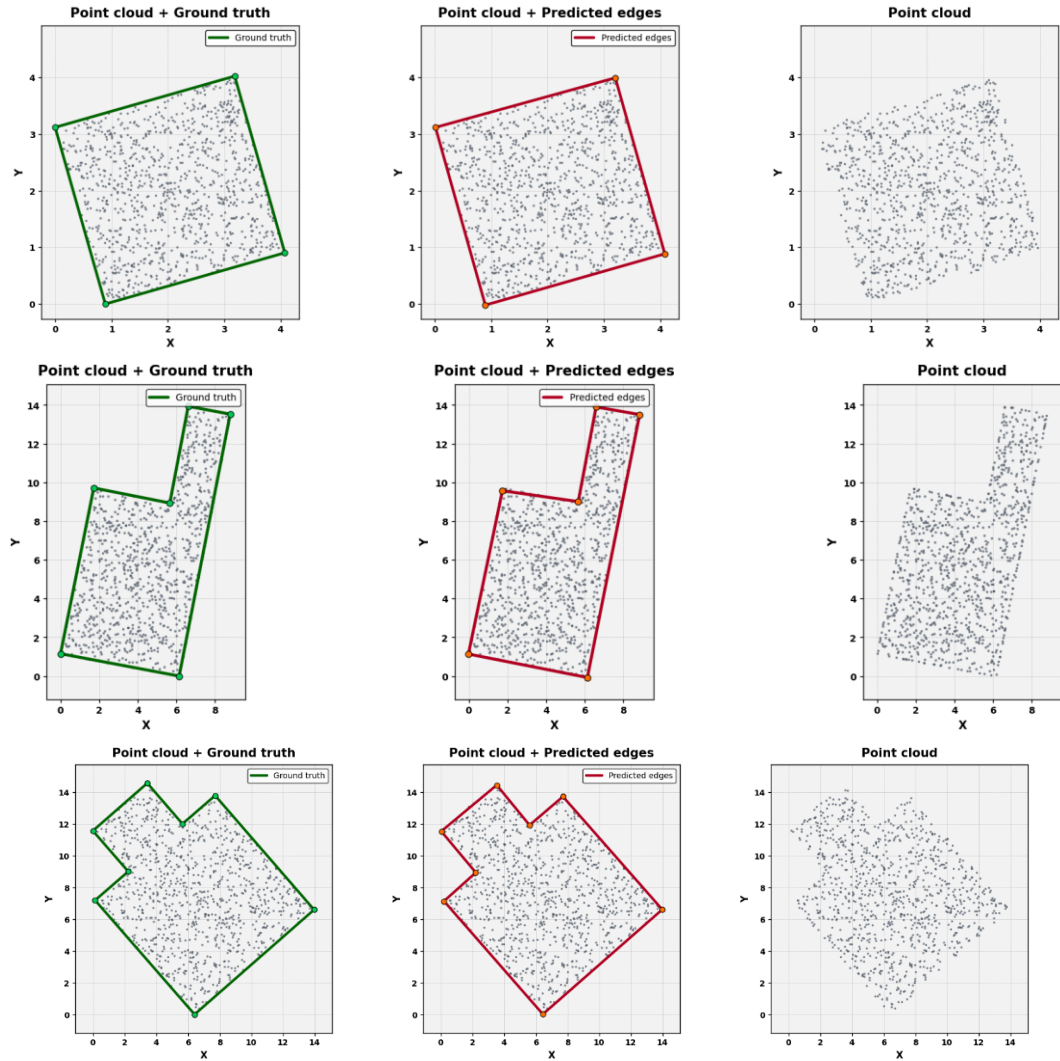
The quantitative results presented in Section 4.3 characterize the average behaviour of the trained model across the held-out test set. To complement these aggregate findings, this section presents individual predictions and inspects the model's behaviour case-by-case. The cases are grouped by building configuration and complexity: standalone components with simple geometries (rectangles, L-, U- and T-shapes), components with more irregular outlines, large standalone components where the 1024-point input budget is spread across a larger boundary, components that share edges with neighbouring buildings (representative of Dutch row-house topology) and finally the principal failure modes observed during inference. For each case, the predicted polygon (red) is overlaid on the BAG reference polygon (green) for visual comparison; for selected examples, the per-stage outputs of the pipeline are shown to illustrate how the prediction emerges through the model.

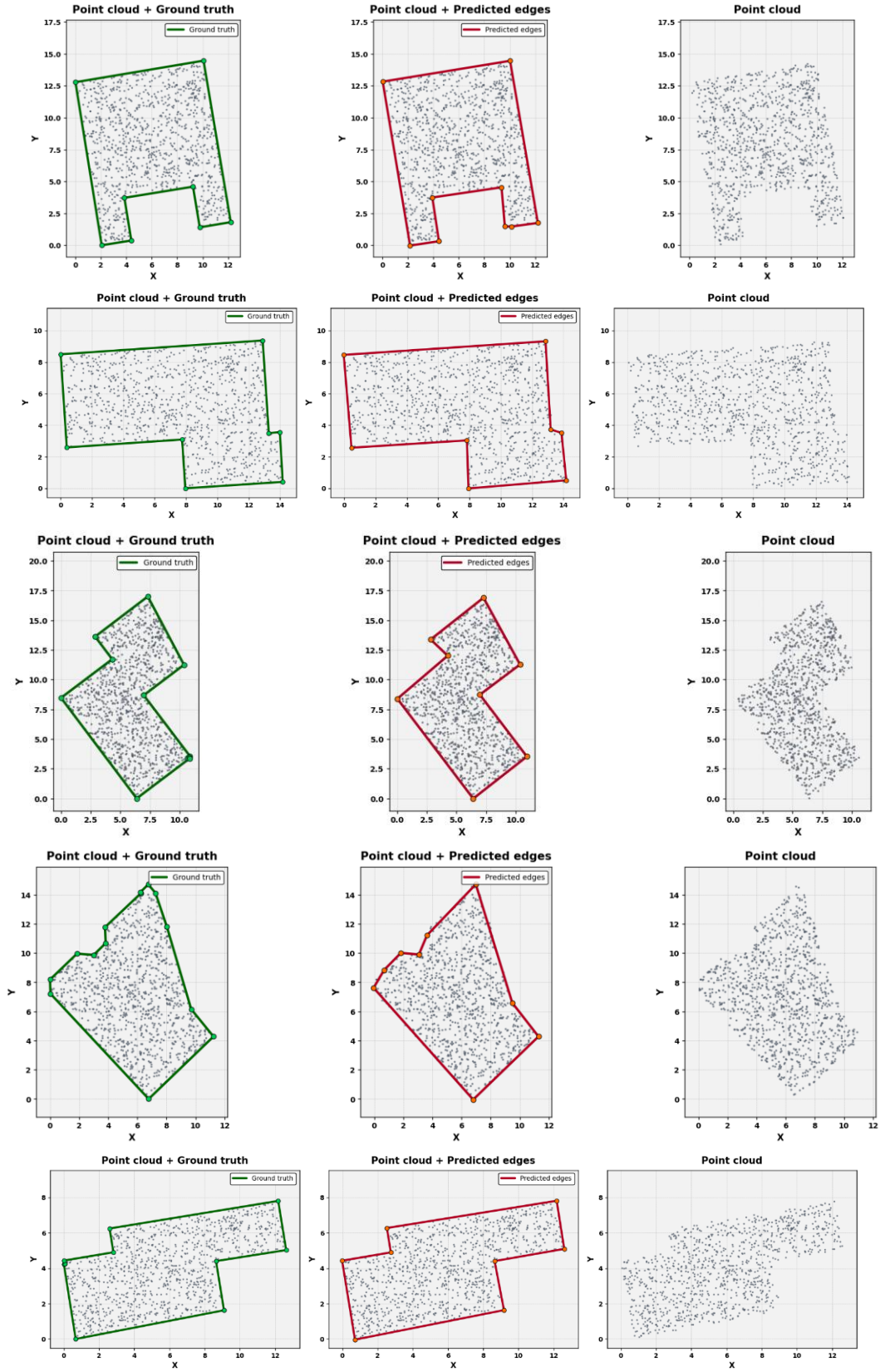
##### **4.4.1. Standalone Building Components**

Standalone building components are the critical topological case for the framework. The building is spatially isolated from its neighbors, the entire boundary corresponds to an exterior wall and the LiDAR points along the boundary have a clear context that supports corner discrimination. The cases in Figure 23 represent a range of geometries with increasing complexity, drawn from this category.

Simple rectangular and L-shape components: for four-vertex rectangular building components, the predicted outline traces the BAG reference almost exactly. The vertices align with the corner positions, the edges follow the wall directions and the polygon is closed and topologically valid without post-processing. L-shaped buildings introduce one concave corner, which is geometrically more demanding than a convex corner because the corner has fewer supporting points along its inner angle. Despite this, the model recovers both convex and concave vertices and orders them correctly along the ring. Small displacements of one or two corners are visible in some examples, but the overall outline structure is preserved. This is the regime in which the architectural pipeline operates as designed: dense corner support across the building, well-separated vertices and a clean topological output.

T-shape, U-shape and more irregular components, as shape complexity increases, the predicted outline continues to follow the BAG reference closely but the agreement between vertex count and vertex localization begins to vary. The T- and U-shape components contain multiple concave corners and longer boundary segments and on these examples the model still recovers the canonical ring order but may displace individual corners slightly along the boundary direction or in a small number of cases, merge two closely spaced corners into a single predicted vertex. This corresponds to the cluster absorption mechanism described in Section 3.2.4: when two ground-truth corners lie within the DBSCAN eps radius of one another after the offset shift, they form a single cluster and produce a single predicted vertex. The visible effect in the figures is that the resulting polygon "cuts the corner" between the two true vertices with a short straight edge, while the rest of the boundary remains accurate. The more irregular components in the lower rows of Figure 23 demonstrate the same pattern, the overall shape is recovered, vertex ordering is consistent but boundary precision degrades modestly as the number of corners increases.





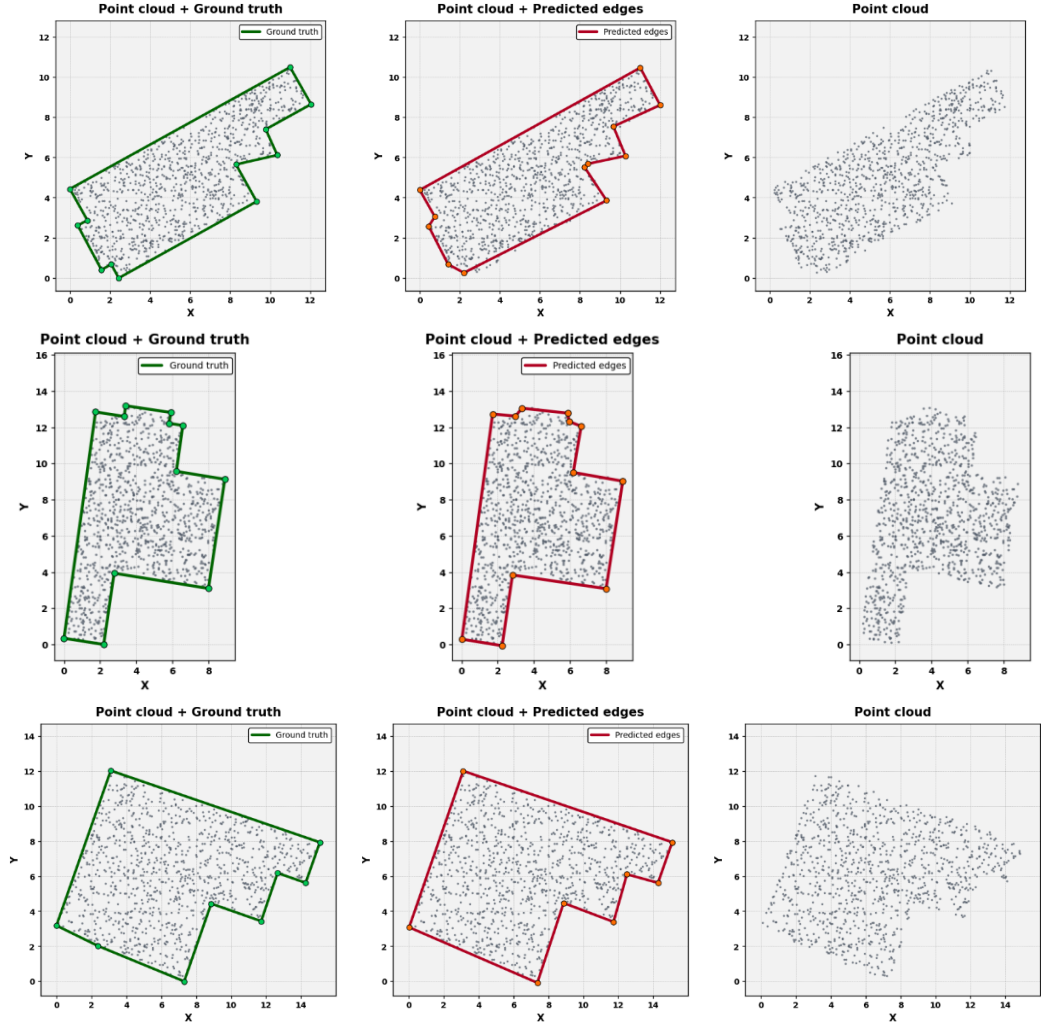


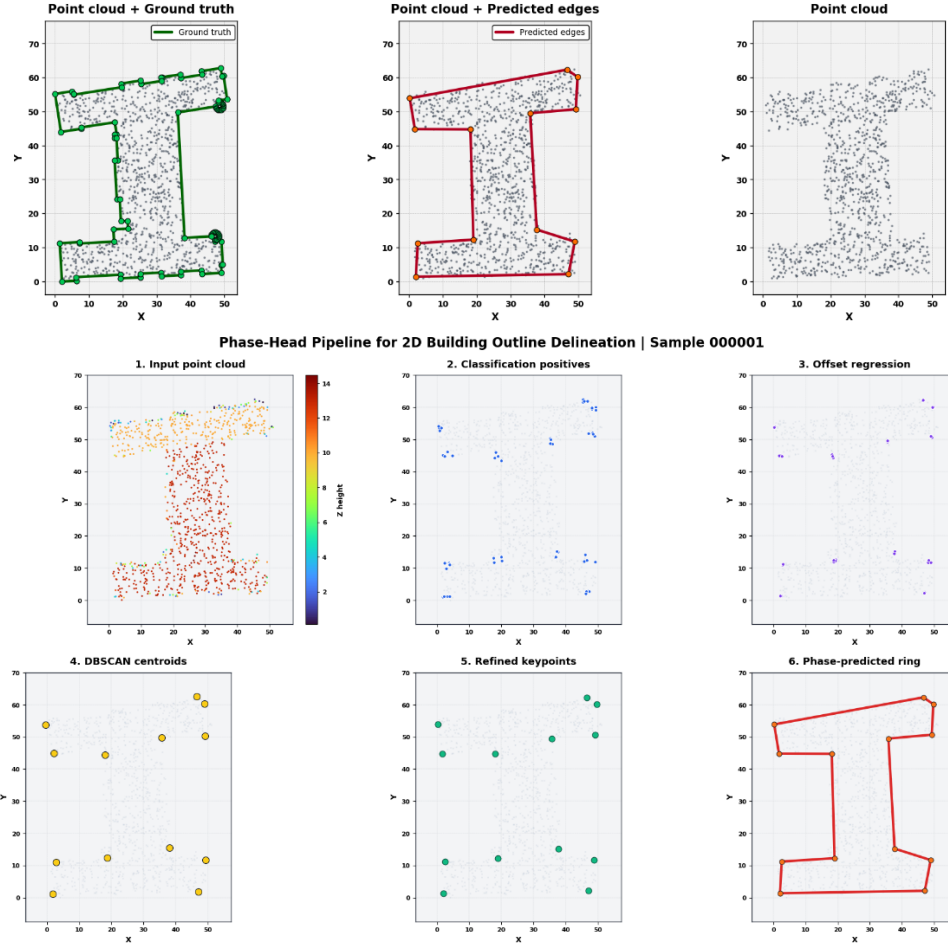
Fig. 24 Successful outline delineation of building components

#### 4.4.2. Large Building Components

Large standalone building components introduce a different challenge. The 1024-point input budget that comfortably covers a regular building is distributed across a substantially larger boundary, reducing the per-corner point density and the discriminative geometric signal available to the classification head. To examine the model's behaviour in this regime, Figure 24 presents two large building components alongside their full pipeline visualizations, showing the per-stage output from input cloud to final ring.

I-shape industrial buildings, the first large component with an I-shape footprint containing 12 reference vertices. The predicted outline (top row of Figure 24) follows the BAG reference closely along all six straight wall segments and the eight predicted vertices that the model recovers are positioned near the corresponding reference vertices.

The smaller extrusions visible in the I-shape's reference polygon include rectangular projections approximately 1-3 m wide and  $\sim 1$  m deep along the northern, southern, and central facades, plus two  $\sim 2$  m diameter circular features on the eastern side. These details are small relative to the building's overall dimensions of  $51 \text{ m} \times 63 \text{ m}$  (each less and comparatively smaller in comparison). The per-stage visualization in the lower portion of the figure shows the geometric reasoning the pipeline produced.



**Fig. 25** Successful outline delineation large standalone building component

In the classification positives panel, the model identifies clusters of points along the boundary corners rather than across the building interior, which is the expected behaviour even when the building is large: the trained corner head has learned to suppress interior surface activations even on a sparsely sampled boundary. The offset regression panel shows these positive points shifted toward their nearest predicted corner, producing tight clusters at most of the building's corners. The DBSCAN centroids panel reduces these clusters to discrete candidate vertices, the refined keypoints panel adjusts each centroid toward its final position, and the phase-predicted ring panel connects these vertices into the closed polygon shown in the top row.

The predicted polygon recovers the I-shape's overall outline correctly but two of the twelve reference corners are absorbed into adjacent clusters. This is visible as straight edges that bridge what should be a small concave step in the building boundary the same cluster-absorption failure mode discussed in Section 4.4.1, manifesting more frequently on this larger component because the wider point spread leaves several genuine corners with marginal support. The case is informative because the failure mode is not catastrophic, that is, the missing vertices reduce the local detail of the outline but do not invalidate the polygon or significantly displace the boundary.

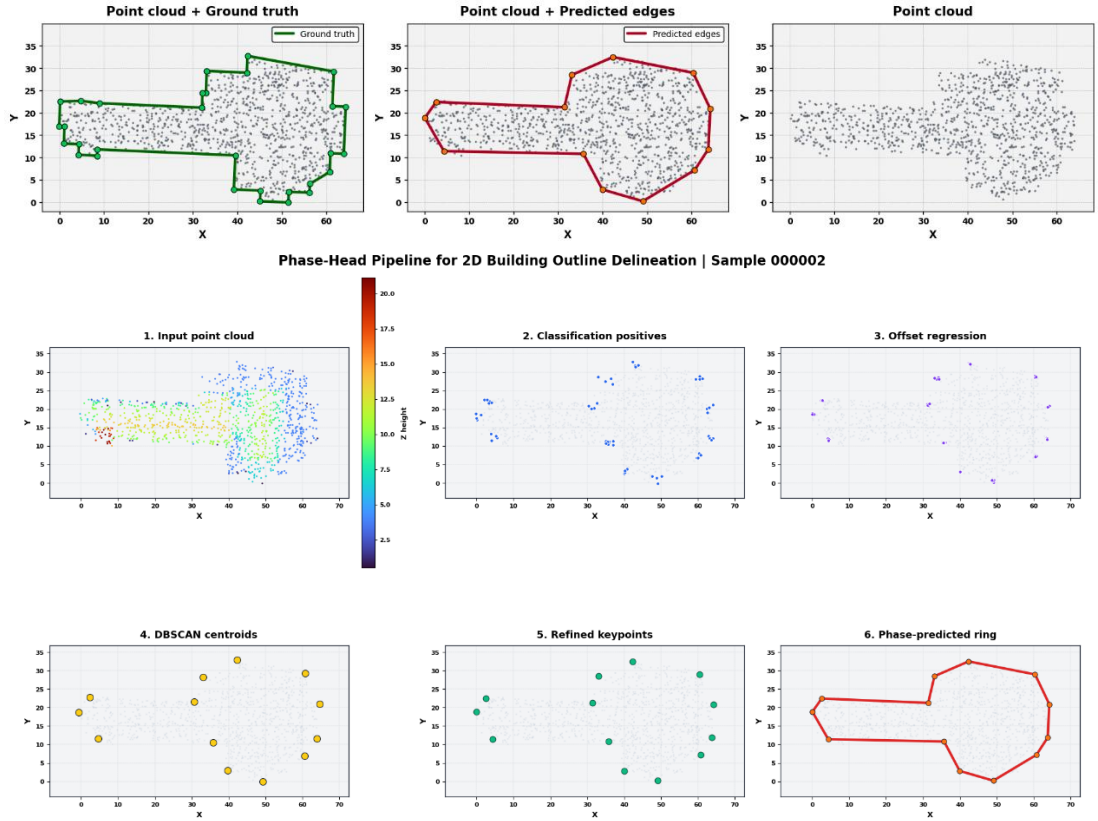
Building with a rounded extension which is the second large component combines a rectangular main body with a curved or polygonal extension on one side. The reference polygon approximates this curvature with a sequence of densely-spaced vertices along the rounded segment.



The predicted outline (top row, right of Figure 25) recovers the rectangular portion accurately and produces a sequence of vertices along the curved extension that approximates the rounded shape. However, the predicted vertex spacing along the curve differs from the reference, and the predicted outline appears smoother than the reference polygon along the rounded segment.

This case illustrates a different aspect of the same point-budget limitation. Where the I-shape example loses vertices to cluster absorption at sharp concave transitions, the rounded-extension example loses vertices along the curve because adjacent reference vertices on a curve are by construction closely spaced relative to eps. The per-stage visualization shows that classification positives and offset shifts concentrate around the curve, but DBSCAN's neighbourhood criterion absorbs adjacent corner evidence into single clusters, producing fewer predicted vertices than the reference contains.

The resulting outline is closer to a regularized polygon approximation of the curve than a vertex-accurate reproduction. Whether this is an acceptable trade-off depends on the downstream application: for cadastral proposal it is sufficient; for exact corner enumeration, additional supervision or a finer clustering criterion would be required.



**Fig. 26** Another successful case of larger component instance

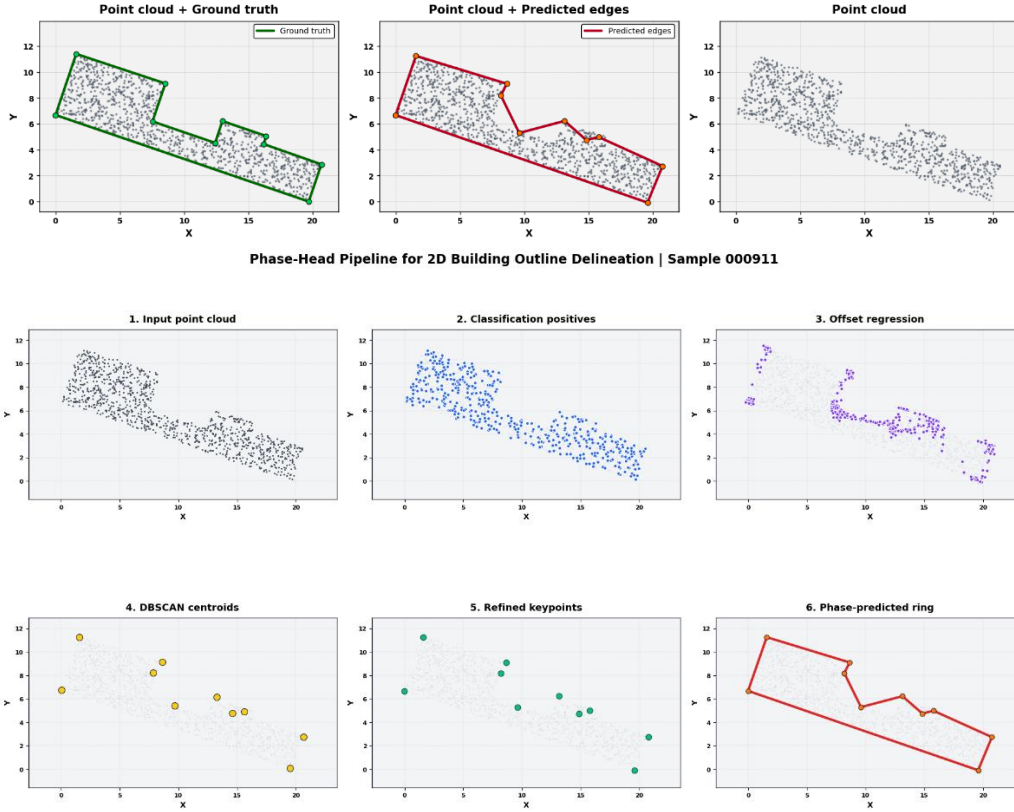
## 4.5. Failure Cases

### 4.5.1. Cluster Absorption in Adjacent Corner

Cluster absorption occurs when two adjacent ground-truth corners are merged into a single predicted vertex during the DBSCAN stage, so that one corner is lost from the outline. The mechanism originates in

the non-differentiable clustering step rather than in the learned heads. When two corners lie close together at a small notch or a closely spaced concave-convex transition, the offset-shifted positive points supporting each corner fall within the same neighbourhood, because the separation between them is smaller than the effective clustering radius ( $\text{eps} = 0.02$  in normalized coordinates). DBSCAN then groups both sets of points into one cluster and its centroid lands between the two true corners rather than at either of them.

The visible symptom is that the polygon "cuts the corner," replacing the two true vertices and the short segment between them with a single straight edge, while the rest of the boundary remains accurate. Unlike the ordering failure, this error cannot be repaired downstream: the refinement head can only adjust a candidate vertex that exists and the phase head can only order the vertices it receives, so once two corners have been absorbed into one cluster the missing vertex is irrecoverable. The failure therefore propagates from the clustering stage as a permanent under-prediction of corners, and is one of the principal contributors to the N-ratio shortfall reported in Section 4.3



**Fig. 27** Failure case (vertex detection)

Figure 26 shows a representative failure case combining two architecturally distinct mechanisms. The building (right panel) is a moderately complex elongated structure with a tilted upper-left block and two small step-up notches along its lower facade. The predicted polygon (middle panel) recovers the correct number of vertices (10) and produces a topologically valid closed ring, but three of the predicted vertices are positioned incorrectly, producing the visible discrepancy with the BAG reference (left panel).

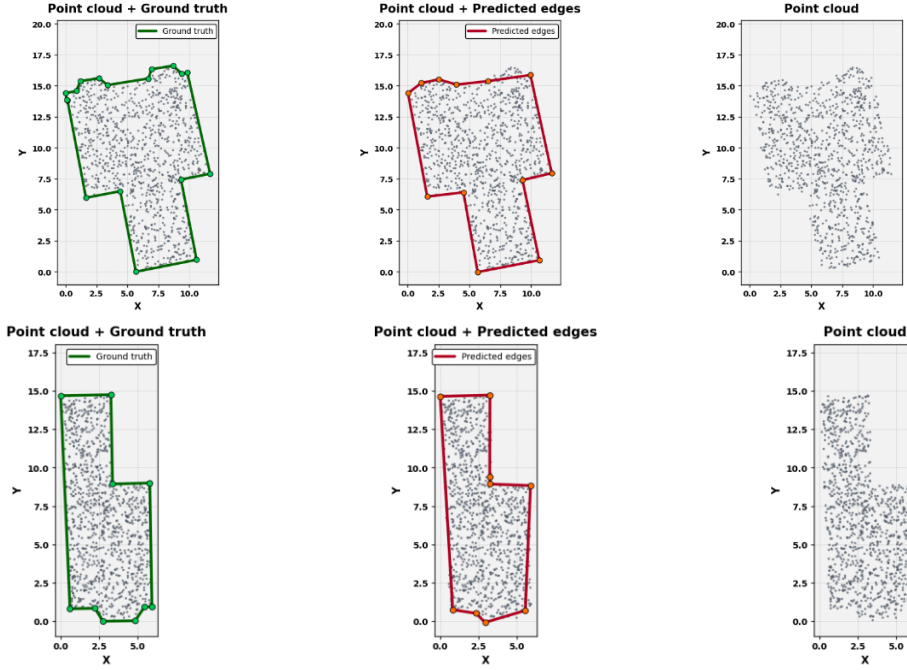
The two small notches along the lower facade are absent from the prediction. Each notch in the GT consists of two corners spaced at a distance apart in world coordinates. This distance falls below the effective DBSCAN clustering radius for a building of this scale ( $\text{eps} = 0.02$  in normalized coordinates,



translating to approximately 0.4 m here). The offset-shifted positive points around each notch's two corners therefore land in overlapping DBSCAN neighborhoods and are merged into a single cluster. The resulting centroid sits between the two true corners rather than at either of them. This is the cluster absorption mechanism: the failure originates in the non-differentiable clustering stage and cannot be recovered by the downstream refinement or phase heads.

A second failure mode is visible in the upper portion of the predicted ring. The GT contains a concave corner, where the upper-left block meets the lower facade. The predicted ring does include a vertex in this region, but it is mis-localized. This is not cluster absorption, since a distinct cluster did form for this region. Instead, this is a vertex mis-localization failure that originates in the offset regression head, that is, the positive candidate corner points around the weak-support concave corner were pulled by their predicted offsets toward the better-supported convex neighbor rather than toward their true target. The resulting cluster centroid lands above the building outline, dragging a visible "kink" into the predicted ring.

The phase head is not implicated in this failure. Given the 10 (incorrectly positioned) vertices it received, the canonical-phase ordering succeeded in producing a closed simple polygon with consistent traversal. The remaining error is entirely geometric, that is, three of the input vertices were at the wrong positions when they reached the phase head. This case illustrates why the per-stage diagnostic outputs are important, without inspecting the DBSCAN centroids and refined key-points, the failure might be misattributed to the phase head, when in fact the architectural fixes that would help are tighter clustering criteria and improved corner discrimination in the offset regression head.



**Fig. 28** Other Cluster absorption cases

All examples in this category trace to the corner-detection pipeline upstream of the phase head: the classification head's score threshold and the DBSCAN clustering criterion. The phase head is not responsible for these failures and cannot repair them. A softer classification criterion (lower threshold) or a tighter clustering criterion (smaller  $\epsilon$ ) each of which carries its own trade-off.

#### 4.5.2. Spurious Vertices

A second category of failure occurs when the corner-classification head produces candidate vertices that do not correspond to any corner of the 2D building outline. The 2D outline is, by definition, the planimetric polygon describing the building's footprint; it contains corners only where the outer walls of the building change direction in the XY plane. Any predicted vertex that does not correspond to such a direction change is spurious from the perspective of outline delineation, regardless of whether the 3D point cloud contains some geometric feature at that location.

The corner-classification head was supervised with ground-truth corners derived from the cadastral BAG polygons, which encode only the planimetric outline of each building. In principle, the head should therefore activate only on points that lie near these outline corners. In practice, the head receives the raw 3D point cloud as input and learns a local geometric criterion for what constitutes a corner. This criterion is such that any point whose local neighborhood resembles the configurations seen at training-time outline corners will receive a high score, even if it does not actually lie on a building outline. As a result, points that lie on internal building elements, such as rooftop structures, internal walls or surface discontinuities not part of the outer footprint, can be classified as corner-related and pass through to the downstream stages.

Once a spurious classification activation occurs, the rest of the pipeline cannot suppress it. DBSCAN forms a cluster around the spurious points just as it would around real outline corner points; the refinement head adjusts the centroid position but has no mechanism to reject a vertex; the phase head receives the spurious vertex along with the legitimate ones and includes it in the sorted ring. The final predicted polygon therefore contains a vertex that does not exist on the cadastral outline. Depending on where the spurious vertex sits, the consequence ranges from a small extra bump along an otherwise straight wall to a long detour that visibly drags the polygon boundary into the interior of the building footprint.

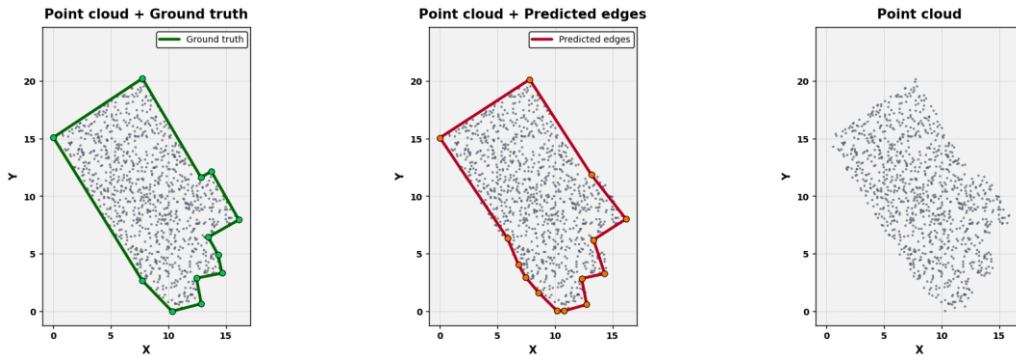


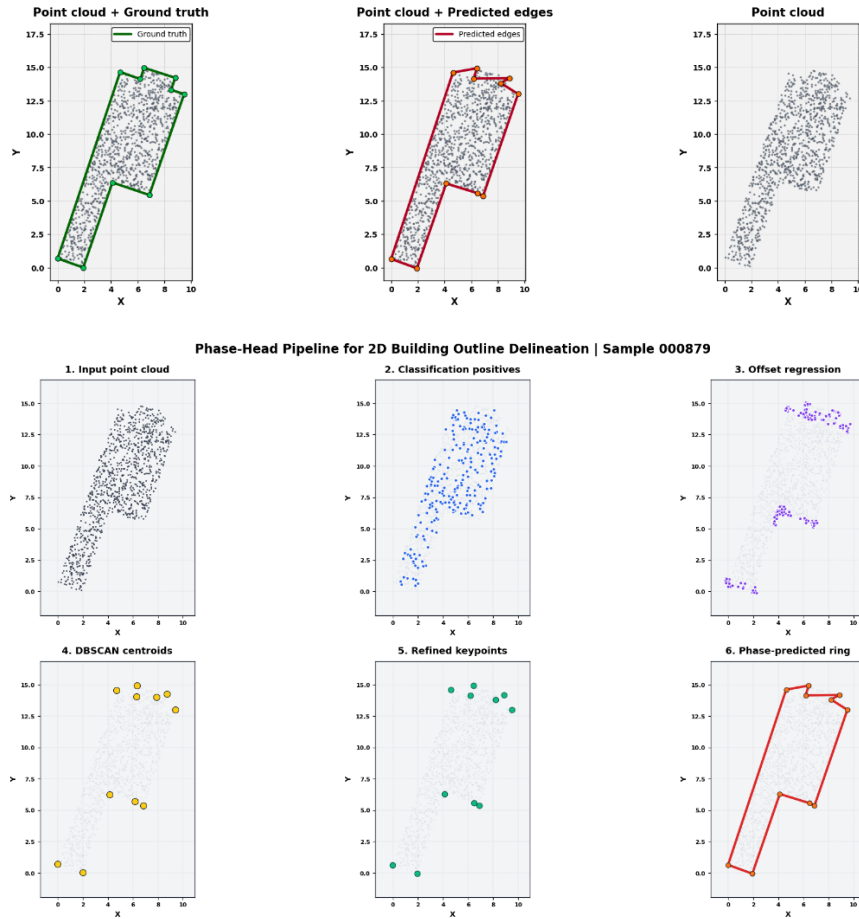
Fig. 29 Spurious vertices case

#### 4.5.3. Ordering Failure (Phase Head)

A wrong-order failure occurs when the candidate vertices are detected and localized correctly but the phase head assigns them inconsistent positions along the ring, so that sorting and connecting consecutive vertices produces a traversal that crosses itself. The mechanism is geometric rather than a localization error. Each vertex's phase is predicted from its refined feature, which is pooled over a local neighbourhood, so when several corners sit close together well inside the pooling radius, as in a tight step structure or when the building has reflective or rotational symmetry, the corners involved draw on nearly the same local context and acquire nearly indistinguishable features. The head then predicts nearly equal

angles for them, the sort order among them becomes unstable, and the ring traverses the affected region non-monotonically.

The visible symptom is one or more boundary crossings, typically an X-shaped or saw tooth-like detour through a corner cluster or across a symmetric pair, while the rest of the outline is reproduced correctly. Because every vertex is present and in the right place, this failure is purely one of ordering; the true outline is recoverable from the same vertex set, and resolving it would require features that distinguish the affected corners by their global position rather than their local geometry alone. The following figure shows a representative phase-head ordering failure on a building component containing a dense cluster of closely-spaced step corners in its upper-right region. The ground-truth polygon (left panel) contains 10 vertices: six along the simple lower body and four forming a tight step structure between approximately.



**Fig. 30** Failure case (wrong vertex ordering and cluster absorption)

The predicted polygon (middle panel) reproduces the lower body of the building correctly but exhibits visible boundary crossings around the upper-right step structure. The diagnostic trace establishes that this case is predominantly a phase-head failure. The pipeline produced eleven refined keypoints; nine of the ten ground-truth corners have a refined keypoint, indicating essentially correct corner detection and refinement.

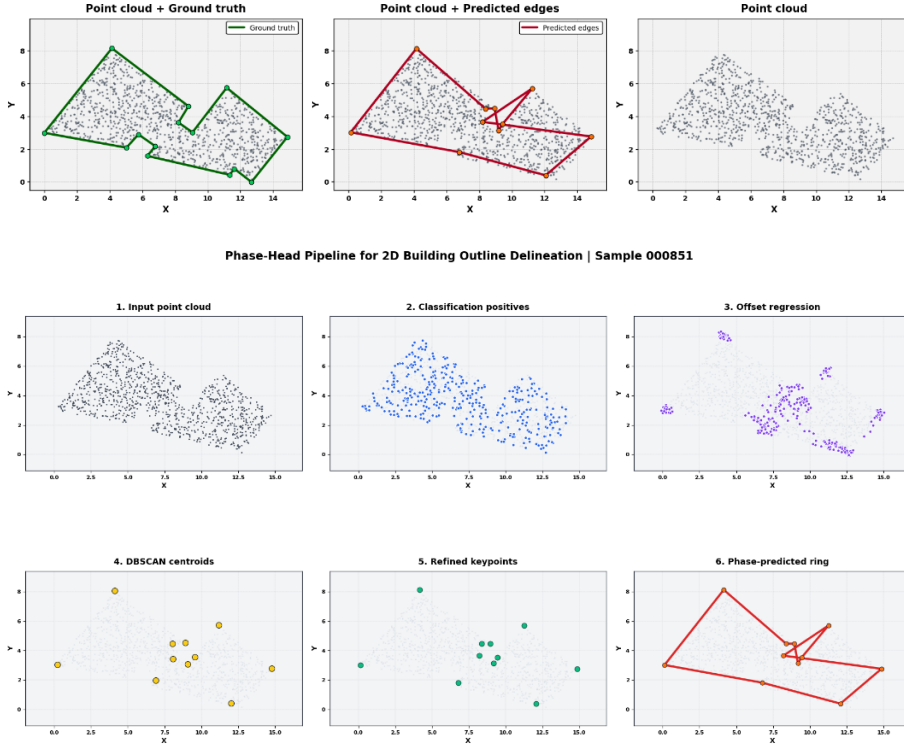
The cause of this ordering failure is geometric. The phase head decides each vertex's position along the boundary using a refined feature, which is computed by pooling information from neighboring points within a certain radius. The four step-cluster vertices sit close together, well inside this pooling radius. The

features for these four vertices therefore draw on largely the same neighborhood and end up looking similar to each other. When the phase head produces a predicted angular position from such similar features, the predictions also come out similar, and the sort order among these vertices becomes unreliable. The ring then traverses the cluster in a non-monotonic order, which produces the visible crossings. The duplicate vertex near the lower notch contributes a second, smaller artefact. Because the duplicate has nearly the same position and nearly the same feature as its neighbor, it receives nearly the same predicted phase. The two vertices end up adjacent in the sorted ring, connected by a very short edge. This appears in the polygon as a small local kink at the duplication site.

#### 4.5.4. Compound Failure (Phase Head)

The final category occurs when the candidate vertices are detected and localized correctly but the phase head assigns swapped canonical positions to two or more of them. After sorting and ring construction, the predicted polygon contains the right vertices but in the wrong topological order, producing self-intersection at one or more locations.

The phase head's per-vertex ordering depends on each vertex having a distinguishable refined feature and when two vertices have indistinguishable features the head cannot reliably choose between equivalent canonical positions. Importantly, resolving this failure would require learned context features from beyond the local neighborhood (a different backbone choice, PointNet++ backbone) or supervised pairing of mirror-symmetric corners during training.



**Fig. 31** Compound Failure case (wrong vertex ordering and cluster absorption)

Figure 31 shows a compound failure on a complex building component combining multiple architectural mechanisms. The ground truth contains 14 vertices, while the pipeline produced only 11 refined keypoints, three vertices were lost to cluster absorption in the bottom-right and lower-left notch regions, where multiple closely-spaced GT corners were merged into single DBSCAN clusters. Conversely, the upper-middle region exhibits the opposite problem; four refined vertices were produced for three GT

corners, indicating local over-segmentation of a single corner cluster or spurious activation on concave features.

The phase head, receiving 11 vertices including duplicate pairs in nearly identical locations, ordered them by canonical angle and placed the duplicates adjacent in the ring. The edges connecting these duplicates produce the visible X-pattern in the middle of the predicted polygon. While this case visually resembles a phase-head ordering failure, the underlying cause is upstream: corruption of the candidate vertex set before the phase head's ordering operation. A correct ordering of an incorrect vertex set cannot recover the true outline.

#### 4.6. Robustness Evaluation

This section examines how the model behaves when the boundary evidence available to it is incomplete, addressing Sub-Objective 3 and research questions 3.1 and 3.2. The way occlusion is treated here follows directly from the pipeline design and must be stated before the results. The model operates on pre-segmented building components, that is, the AHN points are filtered to the building class and connected components are extracted before anything reaches the network (Section 3.2.1). This preprocessing removes the occluding objects themselves such as trees, neighbouring structures, street furniture and together with all surrounding context. Two consequences follow. First, the model is robust by construction to the presence of occluders, because they are never part of its input. Second, and more importantly for this evaluation, naturally occurring occlusion cannot be isolated or quantified at the component level after this stripping which is once a real occluded building is reduced to its surviving building-class points, the only trace of the occlusion is a region of missing boundary returns, with no label for how much was lost or where. Studying real occlusion in this setting would therefore require per-component occlusion-severity annotations that the component-based pipeline does not provide.

For this reason, occlusion is studied here through a controlled synthetic procedure rather than on natural scenes. Following the local-dropping corruption methodology of Ren et al. (2022), synthesising of tree-shadow occlusions by removing points within disc-shaped regions of varying size and number. The disc geometry approximates the plan-view projection of a tree canopy, motivated by domain studies showing that occluded buildings in airborne LiDAR exhibit roughly circular gaps where the canopy blocks the laser pulses (dos Santos et al., 2022; Feng et al., 2020). Three severity levels (light/moderate/heavy) parameterise the number, size, and placement of discs to model the open-residential, narrow-urban progression. Total point count is preserved across severities by densifying the visible region, isolating the spatial-occlusion-pattern variable from per-sample point density."

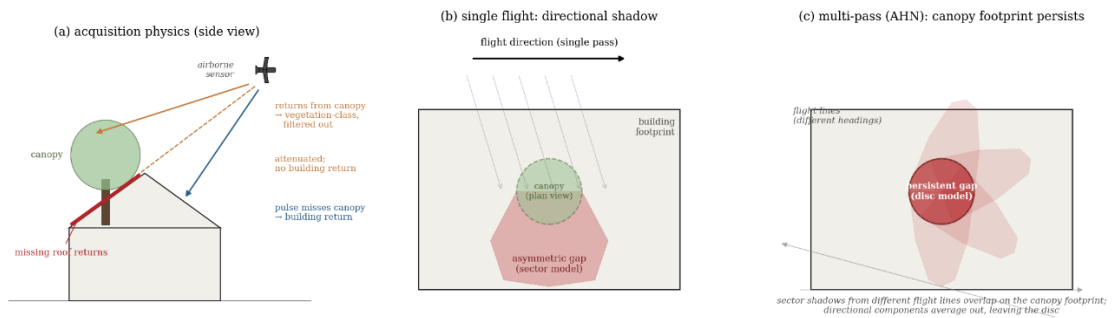
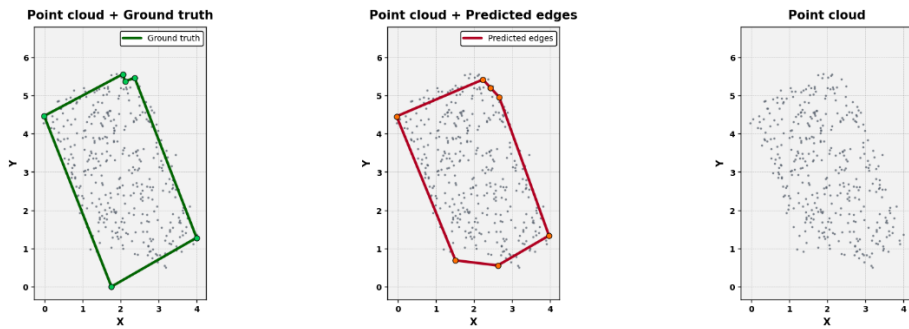


Fig. 32 Illustration of disc-based occlusion model (created with assistance of AI)

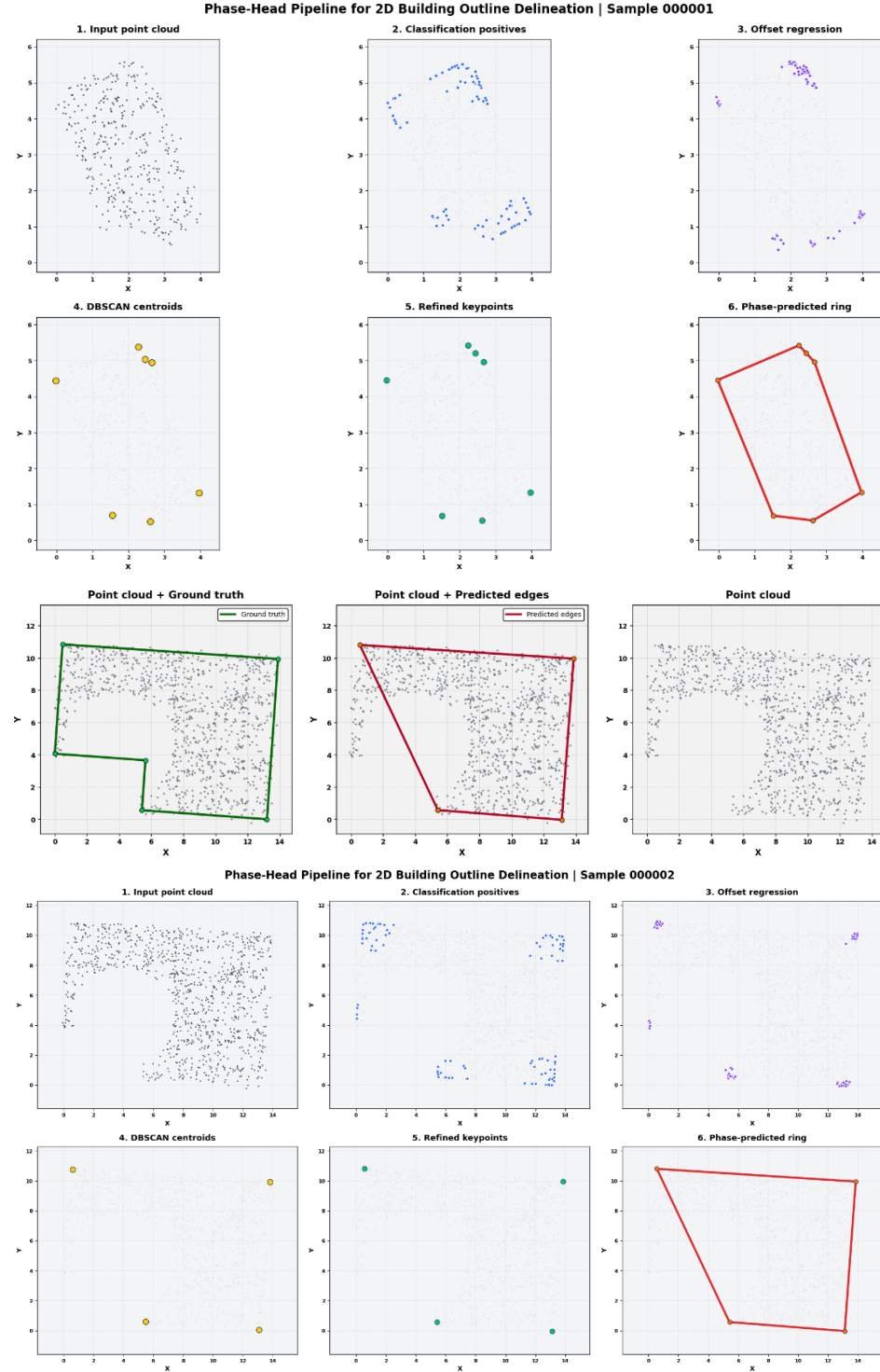
**Synthetic Occlusion Model:** Occlusion in residential AHN scenes is dominated by tree canopy and the disc model is designed to reproduce its effect on the building component rather than the occluder itself. In an airborne acquisition, a pulse directed toward a building that sits beneath a canopy encounters the foliage first. It either returns from the canopy, in which case it is removed as vegetation during class filtering, or it is attenuated and never reaches the building surface. In both cases the building returns beneath the canopy are absent from the building-class component. The geometry of the resulting gap is therefore governed by the plan-view projection of the canopy onto the ground not by the surface that ultimately reflects the pulse. Because the present task is 2D outline delineation, this is the relevant geometry: occlusion presents to the model as a region of the footprint where boundary evidence is missing.

A canopy projects to a compact, approximately convex blob in plan view, which a disc approximates to first order. The disc is deliberately isotropic. An angular-sector model would encode a directional shadow, of the kind produced by strongly oblique occlusion in which the missing region is cast away from the sensor. For airborne LiDAR acquired at modest off-nadir angles, and in the AHN case combined across multiple flight lines and scan directions, the directional component of canopy occlusion largely averages out, and the persistent, acquisition-invariant part of the gap is the footprint of the canopy itself. Modelling occlusion as a disc captures this scan- and direction-invariant component, whereas a sector would impose a directionality that a multi-pass acquisition does not, in general, leave behind.

Uniform random dropout was likewise rejected: it thins support evenly and, in expectation, preserves coverage everywhere along the boundary, so it does not reproduce the property that actually drives the model's failures, namely a contiguous stretch of boundary losing its support so that a corner disappears and the predicted polygon cuts across the gap (the vertex-absent mechanism of Section 4.5.1). The disc removes a localized, contiguous region and reproduces this effect directly.







**Fig. 33** Illustrations of synthetically added occlusion, including inference

The baseline for comparison is the same components evaluated at full point coverage / the main held-out test set of Section 4.3. As in Section 4.3.1, failed predictions are included as zero-overlap in the area-based metrics and excluded from the geometric-distance metrics. The following provides metrics for evaluation with synthetic dataset.

**Table 4** Metrics comparison between full coverage (non-occluded) vs occluded samples

| Metric                           | Full coverage (no occlusions) | Light synthetic occlusion |
|----------------------------------|-------------------------------|---------------------------|
| Precision                        | 0.901                         | 0.864                     |
| Recall                           | 0.884                         | 0.663                     |
| F1 score                         | 0.888                         | 0.738                     |
| IoU                              | 0.862                         | 0.654                     |
| C-IoU                            | 0.784                         | 0.554                     |
| Boundary IoU                     | 0.783                         | 0.525                     |
| Mean PoLiS (successful only)     | 0.180                         | 0.666                     |
| Mean Chamfer (successful only)   | 0.177                         | 0.566                     |
| Mean Hausdorff (successful only) | 0.561                         | 3.602                     |
| Mean N-ratio                     | 0.835                         | 0.713                     |
| Vertex-count recall              | 0.804                         | 0.678                     |
| Exact corner-count match         | 35.7%                         | 16.1%                     |

The most informative observation is the asymmetry between precision and recall. Precision falls only modestly, from 0.901 to 0.864, while recall falls sharply, from 0.884 to 0.663. When points are withdrawn from the boundary, the model continues to place its predicted footprint over genuine building area where it commits to a region, but it misses a substantially larger share of the true footprint. This is the expected response to missing boundary evidence rather than to spurious over-prediction: removing returns along part of the outline strips support from the corners in that region, so they fall below the classification threshold or are absorbed during clustering, and the predicted polygon closes across the gap with a straight edge that cuts away true area. The outcome is lost recall with largely preserved precision.

The effect is concentrated at the boundary rather than spread evenly over the polygon. Among the overlap metrics, Boundary IoU falls the most, from 0.783 to 0.525 and the gap between IoU and Boundary IoU widens from roughly eight percentage points at full coverage to roughly thirteen under occlusion. Removing boundary evidence therefore harms boundary placement disproportionately more than interior overlap, which is consistent with the architecture: the interior is recovered from the surviving vertices, while the boundary band depends on the exact corners that point removal eliminates. The geometric-distance metrics make the worst-case behaviour explicit. Mean Chamfer rises from 0.177 to 0.566 and mean Hausdorff rises from 0.561 to 3.602, so the Hausdorff-to-Chamfer ratio increases from about 3.1 to about 6.4. Much of the boundary still aligns reasonably, but each affected polygon now contains one or more points lying far from the reference, the large cut corners and detours produced where a contiguous stretch of boundary lost its supporting points.

The vertex-count statistics confirm the same mechanism. Mean N-ratio falls from 0.835 to 0.713, so the predicted polygons recover only about 71% of the reference corners on average, and the exact corner-count match rate falls from 35.7% to 16.1%. Withdrawing boundary evidence does not introduce a new failure mode; it amplifies the vertex-absent mechanism already described in Section 4.5.1, in which weakly supported corners are missed at the classification or clustering stage. Under the occlusion condition this



mechanism simply fires more often, because more of the boundary is weakly supported by design. The polygons that are produced nonetheless remain topologically valid: closure and simplicity are guaranteed by construction by the phase head (Section 3.2.3), independently of how much of the boundary the input covers. Synthetic occlusion therefore degrades the geometric and cadastral fidelity of the outline while leaving its topological validity intact, a degraded prediction is still a clean, ingestible polygon.

The vertex-count statistics confirm the same mechanism. Mean N-ratio falls from 0.835 to 0.713, so the predicted polygons recover only about 71% of the reference corners on average, and the exact corner-count match rate falls from 35.7% to 16.1%. Withdrawing boundary evidence does not introduce a new failure mode; it amplifies the vertex-absent mechanism already described in Section 4.5.1, in which weakly supported corners are missed at the classification or clustering stage. Under the occlusion condition this mechanism simply fires more often, because more of the boundary is weakly supported by design. The polygons that are produced nonetheless remain topologically valid: closure and simplicity are guaranteed by construction by the phase head (Section 3.2.3), independently of how much of the boundary the input covers. Synthetic occlusion therefore degrades the geometric and cadastral fidelity of the outline while leaving its topological validity intact; a degraded prediction is still a clean, ingestible polygon.

The interpretation of these results is bounded by the synthetic nature of the experiment. First, synthetic point removal is a proxy for occlusion and may not reproduce its real spatial structure. Real occlusion typically removes a contiguous arc of boundary behind a single occluder and correlates with building height, vegetation proximity and scan geometry, whereas a synthetic procedure reproduces these properties only to the extent that the removal pattern was designed to. The realism of the results therefore depends directly on how the removal was performed, and the conclusions characterize the model's response to that pattern of evidence loss rather than to occlusion in general. Second, only a single light condition is evaluated, so the experiment establishes the direction and mechanism of the effect but not a graded series of severities would be needed to quantify how performance scales as more boundary evidence is withdrawn. Validation against labelled real occluded components remains future work.

Therefore, with respect to RQ3.1 under controlled conditions, the model performs clearly better with complete boundary coverage than with synthetically reduced coverage. Every geometric and cadastral-quality metric degrades when boundary evidence is withdrawn, with the degradation concentrated in recall (0.884 to 0.663), boundary precision (Boundary IoU 0.783 to 0.525) and corner recovery (exact-match rate 35.7% to 16.1%), while precision and topological validity are largely preserved. Because the experiment is synthetic, this answers RQ 3.1 at the level of mechanism: it demonstrates that the difference between lightly and heavily occluded conditions, to the extent occlusion acts by removing boundary support, would be driven by missed boundary regions rather than by false positives. A direct comparison on naturally less- and more-occluded scenes is not possible within the component-based pipeline and is identified as future work.

Occlusion affects the model primarily by removing boundary point support, which propagates through corner detection and clustering as missed vertices and produces large localized boundary errors (mean Hausdorff rising from 0.561 to 3.602 under a light condition). Corner-completeness regularity suffers accordingly but topological regularity does not, because closure and simplicity are architecturally guaranteed regardless of boundary coverage. This is consistent with the known behaviour of LiDAR-based outline methods, in which occlusion reduces boundary completeness and degrades corner fidelity rather than invalidating the extracted geometry. The synthetic design allows the direction and mechanism of this effect to be stated with confidence; quantifying its full extent on real occluded scenes, across multiple severities and with a realistic removal pattern, remains the principal extension of this analysis.

## 5. RESEARCH CHALLENGES

Several practical and methodological challenges were encountered during this research. Among them, the most important challenge was the preparation of reliable training annotations. Since the training labels were derived from BAG polygons, the geometric quality of these polygons directly affected the supervision used by the model. A particularly important issue was the presence of redundant vertices, that is, multiple vertices located along a single straight boundary edge without representing a true corner. In a graph-based learning framework such as the adapted Point2Roof model, such inconsistencies are especially problematic because the model does not only learn building presence but explicit boundary vertices and edge relations. If redundant vertices are retained in the reference data, they can unnecessarily increase graph complexity, distort corner supervision and propagate errors through later steps such as data augmentation and graph-based training. For this reason, correcting the BAG polygons and removing redundant vertices became a major part of the preprocessing effort. This challenge was not only technical, but also methodological because the reliability of the final model depended strongly on the structural consistency of the annotation data.

A second major challenge was the limited point budget used during training and inference. In this research, a fixed point budget was adopted as a deliberate choice in order to keep the process computationally feasible. However, this decision introduced an important trade-off. While it reduced computational cost and made training more practical, it also restricted the amount of geometric evidence available to the model. This limitation became particularly important for larger building components, shared-edge connected cases and outlines with many corners. In such cases, the available points had to represent a wider area and a more complex boundary using the same fixed budget which reduced the local point support available for corner detection and edge reasoning. As a result, preserving valid topology became more difficult as spatial extent and graph complexity increased. This challenge highlights that the performance of the model is influenced not only by architecture and supervision quality but also by the amount of geometric information available within the chosen computational constraints.

## 6. LIMITATIONS

A prominent limitation of this research is that the experiments were conducted on pre-segmented building components rather than on full airborne LiDAR tiles. In the proposed workflow, the input to the model is already reduced to one building component and the target is one corresponding outline graph. This means that the research evaluates the ability of the adapted Point2Roof framework to reconstruct 2D building outlines from isolated building point cloud components but it does not address the broader problem of direct tile-level prediction in mixed urban scenes.

This limitation is important because full LiDAR tiles represent a more realistic operational setting. A tile may contain many buildings, roads, vegetation, terrain variations and other urban objects. In such a setting, a model must first distinguish which points belong to which building and then recover the outline of each building separately. By contrast, the present research removes this scene-level ambiguity in advance through pre-processing. As a result, the study isolates the outline delineation problem from the building detection and instance-separation problem. This was a deliberate methodological choice since the main aim of the thesis was to investigate whether a graph-based deep learning framework can learn valid

2D building outlines from normalized airborne LiDAR point clouds. However, this also means that the conclusions of the study should be interpreted within the scope of component-based outline reconstruction rather than as end-to-end delineation from raw LiDAR tiles.

The use of pre-segmented building components has clear advantages for controlled experimentation. It aligns the input unit with the architecture of Point2Roof which is fundamentally a single-graph predictor. For one building component, the network predicts one set of vertices, one candidate graph and one final outline. This makes the problem more methodologically consistent and allows model behaviour to be analysed more clearly in terms of corner detection, centroid formation, edge classification and graph closure. If full tiles had been used directly, the model would have had to solve two separate problems at the same time; first, instance separation across multiple buildings and second, graph reconstruction for each building. Such a setup would have made it much harder to determine whether failures originated from poor component separation or from poor outline delineation.

At the same time, the component-based setting reduces the realism of the task. Since only building-classified points are retained, the model is not exposed to the full complexity of raw mixed-scene LiDAR data. In practice, neighboring terrain, roads, and other non-building objects may provide contextual information about the true outer extent of a structure. These contextual cues are removed in the present workflow. Therefore, while the adopted setup provides a clean and interpretable evaluation environment, it does not fully represent the challenges of direct tile-level application.

This limitation is further connected to the fixed point budget used in the research. Even in the component-based setting, the limited number of input points already creates difficulty for larger or more complex buildings. In a tile-based setting, the same point budget would have to represent multiple buildings and surrounding scene content at once, which would spread the geometric evidence even more thinly. As a result, corner detection, graph formation, and valid topology recovery would likely become substantially more difficult. This means that extending the present workflow to full-tile prediction would not be a simple scaling step but would require architectural modifications capable of handling multi-instance graph prediction.

For these reasons, the component-based design should be understood as both a strength and a limitation. It is a strength because it allows the outline-delineation problem to be studied in a controlled and scientifically interpretable way. It is a limitation because it does not yet demonstrate full end-to-end building outline extraction from raw airborne LiDAR tiles. Future work should therefore investigate how the adapted Point2Roof framework, or a related graph-based architecture, could be extended toward tile-level inference by combining component separation and outline reconstruction within a unified pipeline.

## 7. FUTURE WORK

Although this research demonstrates that a graph-based deep learning framework can be adapted to delineate 2D building outlines from normalized airborne LiDAR point cloud components, several important directions remain open for future work.

A major direction is the development of a truly end-to-end tile-level framework. In the present study, the model operates on pre-segmented building components which makes the outline-delineation problem more controlled and more compatible with the single-graph design of Point2Roof. However, practical

large-scale deployment would require a framework that can process full airborne LiDAR tiles directly and perform both building instance segmentation and outline delineation within one unified pipeline. Such a framework would need to identify which points belong to individual buildings, separate connected or closely spaced structures and then recover a valid 2D outline for each detected instance.

This would move the workflow from controlled component-based prediction toward a more operational and fully automated setting. From a scientific perspective, this is an important next step because it would allow outline delineation to be evaluated under the true complexity of mixed urban scenes, where buildings, terrain, vegetation and surrounding objects are present simultaneously. In other words, future research should aim not only to reconstruct the outline of a pre-isolated building component but to learn the full sequence from scene interpretation to final polygon generation in one end-to-end design.

A complementary and more incremental direction is to reintroduce the surrounding terrain into the model's input rather than discarding it during preprocessing. In the present workflow only building-classified points are retained (Section 3.2.1), which removes the transition between the building and the surrounding ground. As noted in Chapter 6, this transition is itself informative: the outer footprint boundary is precisely the line at which ground-level returns give way to elevated building returns, so a band of terrain points around each component would supply this boundary cue directly rather than requiring the network to infer the outline from the building points alone. Including such a terrain band as additional input, for instance, the ground points within a fixed buffer around the connected component, would give the model a second, complementary source of evidence for the footprint edge through the building-ground height contrast.

This extension is attractive because it targets the principal weakness identified in this study. Corner recall and boundary completeness degrade wherever per-corner building-point support is thin, on large footprints (Section 4.4.2) and under occlusion (Section 4.6), and in exactly those regions the surrounding ground may still mark where the wall meets the terrain. A model that learns this contrast could therefore recover boundary segments that are weakly supported by building returns alone, potentially easing the recall and N-ratio shortfalls that dominate the current error budget, and it would make a more realistic occlusion study possible by retaining the contextual points that the present setup strips away. Conceptually, introducing terrain also forms a natural bridge between the controlled component-based setting used here and the full tile-level framework described above, since it reintroduces a limited amount of scene context without yet requiring full multi-instance separation. The main design questions it raises, how much terrain to include around each component, how to share the fixed point budget between building and ground returns, and how to supervise the network to attend to the building-ground transition rather than only to building-point corner geometry, are tractable within the existing PointNet++ backbone, which already operates on unordered mixed point sets (Li et al., 2022).

Therefore, extending the present research from a proof of concept for component-based 2D outline learning toward a more general and scalable framework for city-scale building outline delineation from airborne LiDAR.

## 8. CONCLUSION

This thesis set out to design, develop and evaluate a deep learning model that delineates 2D building outlines directly from normalized airborne LiDAR point clouds and to assess its accuracy and cadastral

usability against authoritative BAG footprints under Dutch urban conditions. To this end, the Point2Roof framework was adapted from 3D roof-graph reconstruction to 2D footprint delineation. The central modification replaced the original Paired Point Attention edge classifier with a phase prediction head that assigns each candidate vertex a position along the canonical boundary traversal, so that the outline is recovered by sorting per-vertex angular positions rather than by classifying vertex pairs. The model was trained and evaluated on pre-segmented building components from the AHN dataset, supervised by BAG-derived vertex-edge annotations.

For the first sub-objective (RQ 1.1 and RQ 1.2), the study showed that 2D footprint vertices and edges can be recovered from normalized LiDAR by combining point-wise corner classification and offset regression, density-based clustering and refinement of candidate vertices and per-vertex phase ordering. The most effective architectural modification for producing clean, topologically valid outlines was the phase head itself. Because boundary connectivity in a closed polygon is fully determined by the order of its vertices, replacing pairwise edge classification with per-vertex ordering brings the predicted topology into exact alignment with the target output, that is, every polygon the pipeline produces is a simple closed ring by construction. This eliminates the disconnected-edge, vertex-degree and open-ring failures that an independent edge classifier can produce and removes the need for a separate closure or cycle-detection step.

For the second sub-objective (RQ 2.1 and RQ 2.2), the predicted outlines matched BAG footprints closely on average, an IoU of 0.862, an F1 of 0.888, a Boundary IoU of 0.783, and a mean Chamfer distance of 0.177 in normalized units, with the principal geometric limitation being localized worst-case error at individual corners rather than systematic shape distortion, reflected in the comparatively larger Hausdorff distance. On cadastral quality, the outlines satisfy topological correctness unconditionally, since simplicity and closure are guaranteed by the phase head; the only departures from valid output were the small fraction of components (about 2%) that failed to yield a usable polygon at all. The main shortfall is vertex completeness, that is, the predicted polygons recover roughly 84% of the reference corners on average (mean N-ratio 0.835) and reproduce the exact corner count in about a third of cases, because weakly supported corners are missed at the classification or clustering stage and the boundary "cuts the corner" between the surrounding detected vertices. The outlines are therefore suitable for outline proposal and human-supervised review but not yet for the centimetre-level precision required for fully automated cadastral entry.

For the third sub-objective (RQ 3.1 and RQ 3.2), robustness was examined under occlusion. Because the pipeline operates on pre-segmented components, occluding objects are removed before inference, so the model is robust by construction to their presence; what occlusion actually leaves behind is missing boundary evidence. This was studied through a controlled synthetic disc-removal model emulating the plan-view footprint of tree-canopy shadows. Under reduced coverage, precision and topological validity were largely preserved while recall, boundary precision, and corner recovery fell sharply and worst-case boundary error rose several-fold. Occlusion therefore does not introduce a new failure mode but amplifies the existing vertex-absent mechanism, degrading the geometric fidelity of the outline while leaving its topological validity intact. Because the experiment is synthetic and limited in severity range, this answers the third research question at the level of mechanism rather than as a full characterization on real occluded scenes.

The contribution of this thesis is consequently not a new network but the adaptation, supervision design and critical evaluation of an existing graph-based LiDAR framework for 2D cadastral-style outline delineation. The phase head is the principal methodological contribution which is a per-vertex ordering

formulation that makes closure and topological validity structural properties of the output rather than learned constraints and that scales linearly rather than quadratically with the number of vertices. The accompanying failure-mode analysis consisting of distinguishing cluster absorption, vertex mislocalization, spurious vertices and ordering errors and attributing each to a specific stage of the pipeline, is itself a result, since it converts aggregate metrics into actionable architectural diagnoses.

A recurring question is whether this learned approach is more robust to varying point density than the traditional fixed-parameter  $\alpha$ -shape pipeline of Dorninger and Pfeifer (2008). The two are robust in different ways. The  $\alpha$ -shape depends on a single fixed metric parameter whose appropriate value changes with local point density (dos Santos et al., 2018), so one setting rarely suits an entire dataset. The present method has no such global parameter: per-sample normalization and a fixed point budget express its spatial parameters relative to each building's own extent, so the same configuration applies without re-tuning across buildings of different size and acquisition density. In that specific sense it is structurally more robust. The advantage is not unconditional, however, since the model remains sensitive to thin per-corner support, as the large-building and occlusion results show, whereas a fixed  $\alpha$  fails on those same sparse regions for a different reason, no single  $\alpha$  can both preserve fine detail in densely sampled buildings and bridge gaps in sparsely sampled ones. The two methods therefore differ in their density-failure signature rather than in a simple ordering and a controlled density-stratified comparison would be required to make this contrast quantitative.

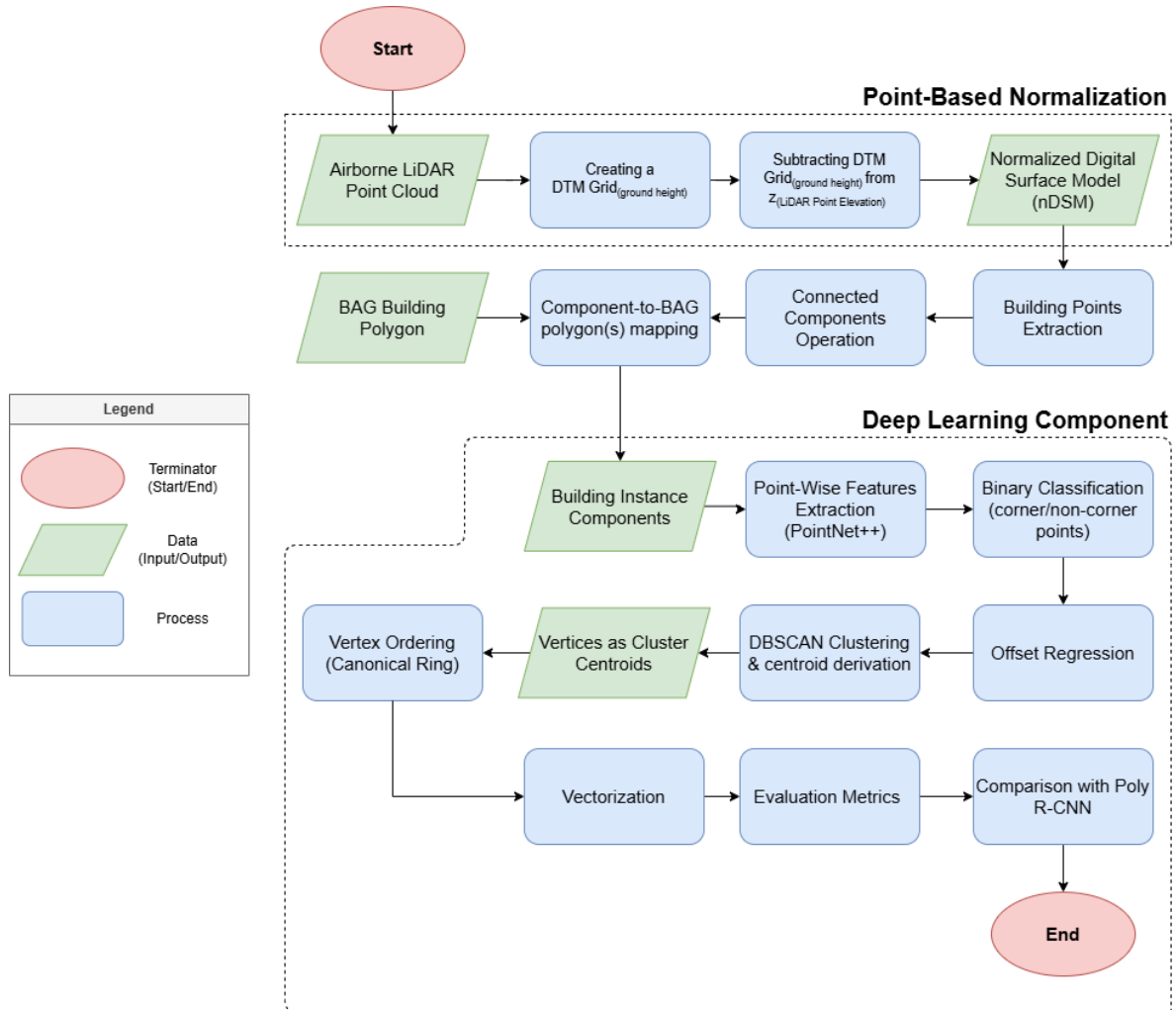
These findings hold within a deliberately controlled scope. The model was trained and evaluated on pre-segmented components rather than full LiDAR tiles, on samples curated to a clean one-to-one correspondence between component and outline and under a fixed point budget choices that isolate the outline-delineation problem but exclude instance separation, ambiguous or temporally inconsistent buildings and densely sampled large-footprint cases (Chapters 5 and 6). The reported metrics should accordingly be read as describing the favourable, well-formed end of the operational range. The clearest directions for future work (Chapter 7) follow directly from these boundaries: extending the framework to tile-level inference with joint instance separation and outline reconstruction; relaxing the one-to-one curation to handle merged, fragmented and changed buildings, the last connecting naturally to cadastral change detection; increasing or adapting the point budget to relieve the per-corner support limitation; and validating the occlusion behaviour on real, severity-labelled scenes.

In sum, this work provides evidence that a graph-based framework originally built for 3D roof reconstruction can be redirected to produce structurally valid, GIS-ready 2D building outlines directly from normalized airborne LiDAR, without the hand-tuned geometric parameters and post-hoc regularization that traditional pipelines require. Its outputs are topologically clean by construction and geometrically faithful for the typical building, with limitations that are well-localized and architecturally interpretable. It therefore stands as a proof of concept for component-level cadastral outline delineation and a concrete foundation for the larger goal of automated, city-scale building-outline extraction from airborne LiDAR.

**Disclaimer:** I acknowledge the use of AI, in the development of this work. AI was utilized for brainstorming, refining ideas, rephrasing sentences and enhancing clarity while maintaining the originality and coherence of the content. All final edits, interpretations and conclusions are my own.

## 9. ANNEXES

### 9.1. Annex A – Workflow Diagram





## LIST OF REFERENCES

---

- Actueel Hoogtebestand Nederland (AHN). (2021). AHN3: Actueel Hoogtebestand Nederland. <https://www.ahn.nl>
- Anjanappa, G., Elberink, S. O., Maiti, A., Lin, Y., & Vosselman, G. (2026). Map2ImLas: Large-scale 2D-3D airborne dataset with map-based annotations. *ISPRS Open Journal of Photogrammetry and Remote Sensing*, 19, 100112. <https://doi.org/10.1016/j.ophoto.2025.100112>
- Buyukdemircioglu, M., Can, R., Kocaman, S., & Kada, M. (2022). Deep Learning Based Building Footprint Extraction from Very High Resolution True Orthophotos and nDSM. *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, V-2-2022, 211-218. XXIV ISPRS Congress “Imaging today, foreseeing tomorrow”, Commission II - 2022 edition, 6 11 June 2022, Nice, France. <https://doi.org/10.5194/isprs-annals-V-2-2022-211-2022>
- Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., & Zagoruyko, S. (2020). End-to-End Object Detection with Transformers. In A. Vedaldi, H. Bischof, T. Brox, & J.-M. Frahm (Eds.), *Computer Vision – ECCV 2020* (pp. 213–229). Springer International Publishing. [https://doi.org/10.1007/978-3-030-58452-8\\_13](https://doi.org/10.1007/978-3-030-58452-8_13)
- Castrejon, L., Kundu, K., Urtasun, R., & Fidler, S. (2017). Annotating Object Instances with a Polygon-RNN (arXiv:1704.05548). *arXiv*. <https://doi.org/10.48550/arXiv.1704.05548>
- Dorninger, P., & Pfeifer, N. (2008). A Comprehensive Automated 3D Approach for Building Extraction, Reconstruction, and Regularization from Airborne Laser Scanning Point Clouds. *Sensors*, 8(11), 7323-7343. <https://doi.org/10.3390/s8117323>
- dos Santos, R. C., Galo, M., & Carrilho, A. C. (2018). BUILDING BOUNDARY EXTRACTION FROM LIDAR DATA USING A LOCAL ESTIMATED PARAMETER FOR ALPHA SHAPE ALGORITHM. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, XLII-1, 127-132. ISPRS TC I Mid-term Symposium Innovative Sensing - From Sensors to Methods and Applications (Volume XLII-1) - 10;12 October 2018, Karlsruhe, Germany. <https://doi.org/10.5194/isprs-archives-XLII-1-127-2018>
- dos Santos, R. C., Habib, A. F., & Galo, M. (2022). Weighted Iterative CD-Spline for Mitigating Occlusion Effects on Building Boundary Regularization Using Airborne LiDAR Data. *Sensors*, 22(17), 6440. <https://doi.org/10.3390/s22176440>
- Douglas, D. H., & Peucker, T. K. (1973). Algorithms for the reduction of the number of points required to represent a digitized line or its caricature. *Cartographica*, 10(2), 112–122. <https://doi.org/10.3138/FM57-6770-U75U-7727>
- Fan, H., Su, H., & Guibas, L. (2017). A Point Set Generation Network for 3D Object Reconstruction from a Single Image. 2463–2471. <https://doi.org/10.1109/CVPR.2017.264>
- Feng, M., Zhang, T., Li, S., Jin, G., & Xia, Y. (2020). An improved minimum bounding rectangle algorithm for regularized building boundary extraction from aerial LiDAR point clouds with partial

occlusions. *International Journal of Remote Sensing*, 41(1), 300–319. <https://doi.org/10.1080/01431161.2019.1641245>

Gilani, S. A. N., Awrangjeb, M., & Lu, G. (2016). An automatic building extraction and regularisation technique using LiDAR point cloud data and orthoimage. *Remote Sensing*, 8(3), 258. <https://doi.org/10.3390/rs8030258>

Girshick, R. (2015). Fast R-CNN. 2015 IEEE International Conference on Computer Vision (ICCV), 1440–1448. <https://doi.org/10.1109/ICCV.2015.169>

Hattula, E., Zhu, L., & Raninen, J. (2024). Building extraction in urban and rural areas with aerial and LiDAR DSM. *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, X-4-W4-2024, 73-79. ISPRS WG IV/9 8th International Conference on Smart Data and Smart Cities (SDSC) - 4-7 June 2024, Athens, Greece. <https://doi.org/10.5194/isprs-annals-X-4-W4-2024-73-2024>

Hu, Y., Wang, Z., Huang, Z., & Liu, Y. (2023). PolyBuilding: Polygon transformer for building extraction. *ISPRS Journal of Photogrammetry and Remote Sensing*, 199, 15-27. <https://doi.org/10.1016/j.isprsjprs.2023.03.021>

Huang, S., Wang, R., Guo, B., & Yang, H. (2024). PBWR: Parametric-Building-Wireframe Reconstruction from Aerial LiDAR Point Clouds. 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 27778-27787. <https://doi.org/10.1109/CVPR52733.2024.02624>

Huttenlocher, D. P., Klanderman, G. A., & Rucklidge, W. A. (1993). Comparing Images Using the Hausdorff Distance. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 15(09), 850–863. <https://doi.org/10.1109/34.232073>

Ji, Y., Wu, W., Wan, G., Zhao, Y., Wang, W., Yin, H., Tian, Z., & Liu, S. (2024). Segment Anything Model-Based Building Footprint Extraction for Residential Complex Spatial Assessment Using LiDAR Data and Very High-Resolution Imagery. *Remote Sensing*, 16(14), 2661. <https://doi.org/10.3390/rs16142661>

Jiao, W., Persello, C., & Vosselman, G. (2024). PolyR-CNN: R-CNN for end-to-end polygonal building outline extraction (No. arXiv:2407.14912). arXiv. <https://doi.org/10.48550/arXiv.2407.14912>

Kadaster. (2023). Basisregistratie Adressen en Gebouwen (BAG). <https://www.kadaster.nl/bag>

Kuhn, H. W. (1955). The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2), 83–97. <https://doi.org/10.1002/nav.3800020109>

Li, L., Song, N., Sun, F., Liu, X., Wang, R., Yao, J., & Cao, S. (2022). Point2Roof: End-to-end 3D building roof modeling from airborne LiDAR point clouds. *ISPRS Journal of Photogrammetry and Remote Sensing*, 193, 17-28. <https://doi.org/10.1016/j.isprsjprs.2022.08.027>

Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection (pp. 2980-2988). <https://doi.org/10.1109/ICCV.2017.324>

Liu, Y., Obukhov, A., Wegner, J. D., & Schindler, K. (2024). Point2Building: Reconstructing buildings from airborne LiDAR point clouds. *ISPRS Journal of Photogrammetry and Remote Sensing*, 215, 351-368. <https://doi.org/10.1016/j.isprsjprs.2024.07.012>

Liu, Y., Zhu, L., Ye, H., Huang, S., Gao, X., Zheng, X., & Shen, S. (2025). BWFormer: Building Wireframe Reconstruction from Airborne LiDAR Point Cloud with Transformer. 2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 22215-22224. <https://doi.org/10.1109/CVPR52734.2025.02069>

Liu, J., Skidmore, A. K., Heurich, M., & Wang, T. (2017). Significant effect of topographic normalization of airborne LiDAR data on the retrieval of plant area index profile in mountainous forests. *ISPRS Journal of Photogrammetry and Remote Sensing*, 132, 77-87. <https://doi.org/10.1016/j.isprsjprs.2017.08.005>

Manders, N. (2023). Comparing AHN point clouds for their performance in representing 3D buildings in Zuid-Holland: A quantitative and qualitative performance review between AHN3 and AHN4 (Master's thesis, Utrecht University, The Netherlands). GDMC. <https://www.gdmc.nl/publications/2023/MScThesisNiekManders.pdf>

Martin Ester, H.-P. K. (n.d.). A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise. AAAI. Retrieved June 5, 2026, from <https://aaai.org/papers/kdd96-037-a-density-based-algorithm-for-discovering-clusters-in-large-spatial-databases-with-noise/>

Mohamed, S. A., Mahmoud, A. S., Moustafa, M. S., Helmy, A. K., & Nasr, A. H. (2022). Building footprint extraction in dense areas from LiDAR data using Mask R-CNN. *International Journal of Advanced Computer Science and Applications*, 13(6). <https://doi.org/10.14569/IJACSA.2022.0130643>

Oude Elberink, S. J., & Maas, H. G. (2000). The use of anisotropic height texture measures for the segmentation of airborne laser scanner data. In *Proceedings of 4 the 19th International Congress for Photogrammetry and Remote Sensing* (Vol. 33, Part B3/2, pp. 678-684). International Society for Photogrammetry and Remote Sensing.

Preparata, F. P., & Shamos, M. I. (1985). *Computational geometry: An introduction*. Springer-Verlag

Publieke Dienstverlening Op de Kaart (PDOK). (2023). PDOK aerial orthophotos. <https://www.pdok.nl>

Ren, J., Pan, L., & Liu, Z. (2022). Benchmarking and Analyzing Point Cloud Classification under Corruptions. <https://doi.org/10.48550/arXiv.2202.03377>

Rottmann, P., Haunert, J.-H., & Dehbi, Y. (2022). Automatic Building Footprint Extraction from 3D Laserscans. *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, X-4-W2-2022, 233-240. *ISPRS TC IV 17th 3D GeoInfo Conference - 19&ndash;21 October 2022, Sydney, Australia*. <https://doi.org/10.5194/isprs-annals-X-4-W2-2022-233-2022>

Šanca, S., Jyhne, S., Gazzea, M., & Arghandeh, R. (2023). AN END-TO-END DEEP LEARNING WORKFLOW FOR BUILDING SEGMENTATION, BOUNDARY REGULARIZATION AND VECTORIZATION OF BUILDING FOOTPRINTS. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, XLVIII-4-W7-2023, 169-175. *ISPRS ICWG IV/III/IIIFree and Open Source Software for Geospatial (FOSS4G) 2023 - Academic Track - 26*

June&ndash;2 July 2023, Prizren, Kosovo. <https://doi.org/10.5194/isprs-archives-XLVIII-4-W7-2023-169-2023>

Schuegraf, P., Li, Z., Tian, J., Shan, J., & Bittner, K. (2024). Rectilinear Building Footprint Regularization Using Deep Learning. *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, X-2-2024, 217-222. ISPRS TC II Mid-term Symposium “The Role of Photogrammetry for a Sustainable World” - 11-14 June 2024, Las Vegas, Nevada, USA. <https://doi.org/10.5194/isprs-annals-X-2-2024-217-2024>

Sun, X., Zhao, W., Maretto, R. V., & Persello, C. (2021). Building Outline Extraction From Aerial Imagery And Digital Surface Model With a Frame Field Learning Framework. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, XLIII-B2-2021, 487-493. XXIV ISPRS Congress Imaging today, foreseeing tomorrow, Commission II - 2021 edition, 5 9 July 2021. <https://doi.org/10.5194/isprs-archives-XLIII-B2-2021-487-2021>

Qi, C. R., Yi, L., Su, H., & Guibas, L. J. (2017, June 7). PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space. *arXiv.Org*. <https://arxiv.org/abs/1706.02413v1>

Wang, C., Chen, J., Meng, Y., Deng, Y., Li, K., & Kong, Y. (2024). SAMPolyBuild: Adapting the Segment Anything Model for polygonal building extraction. *ISPRS Journal of Photogrammetry and Remote Sensing*, 218, 707-720. <https://doi.org/10.1016/j.isprsjprs.2024.09.018>

Zhou, Y., Barnes, C., Lu, J., Yang, J., & Li, H. (2019). On the Continuity of Rotation Representations in Neural Networks. 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 5738–5746. <https://doi.org/10.1109/CVPR.2019.00589>

Zorzi, S., Bazrafkan, S., Habenschuss, S., & Fraundorfer, F. (2022). PolyWorld: Polygonal Building Extraction with Graph Neural Networks in Satellite Images. 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 1938–1947. <https://doi.org/10.1109/CVPR52688.2022.00189>

